



SAINT PETER'S UNIVERSITY

School of Global Innovation and AI

DS-620-F1

Data Visualization

WEEKS 3 & 4

**Where things go on the page,
and what happens when you touch them**

Reda MASTOURI

rmastouri@saintpeters.edu

We are still answering the same three questions

Week 1 gave us the three questions. Week 2 filled in two of them.

WHY is someone looking? We have the verb-and-noun vocabulary for that.

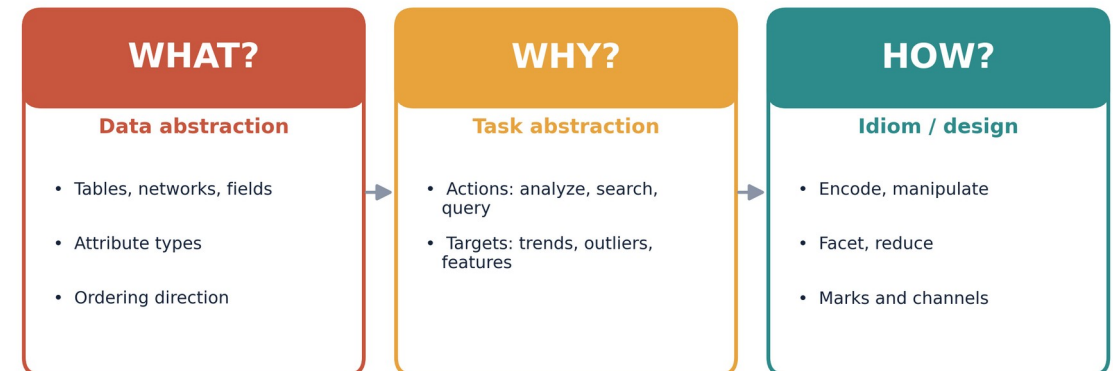
HOW is it drawn? We know a chart is shapes plus ways of varying them.

Tonight goes deeper into HOW. Marks and channels told us what a single shape can carry. Now we ask where to put those shapes, and what happens when the reader can move them.

IN ONE LINE

Tonight is still the HOW question — just one level down, in the details.

Munzner, Visualization Analysis and Design, Chapter 1.



Munzner's analysis framework: every visualization answers all three questions

Two weeks, one session

3 WEEK 3 — Arranging tables

Given a table, where does everything go?

How many key columns you have decides what you can draw. Then the three moves that make a bar chart work, the rules for lines and axes, and what happens when you bend the axes into a circle.

Ends with the charts people reach for when there are too many columns.

4 WEEK 4 — Interaction & views

What changes when the reader can touch it?

Three ways a single view can change, three ways to use more than one view, and the evidence about when each one helps.

Ends with the rules of thumb for building something people can actually use.

Break in the middle, between the two halves.

Where tonight sits

We are covering the Weeks 3 and 4 material in one sitting.

After tonight the theory is essentially complete. Week 5 starts Tableau, and the project starts in Week 6.

The midterm in Week 7 draws on everything from Weeks 1 to 4 — so tonight is the last big block of examinable concepts.

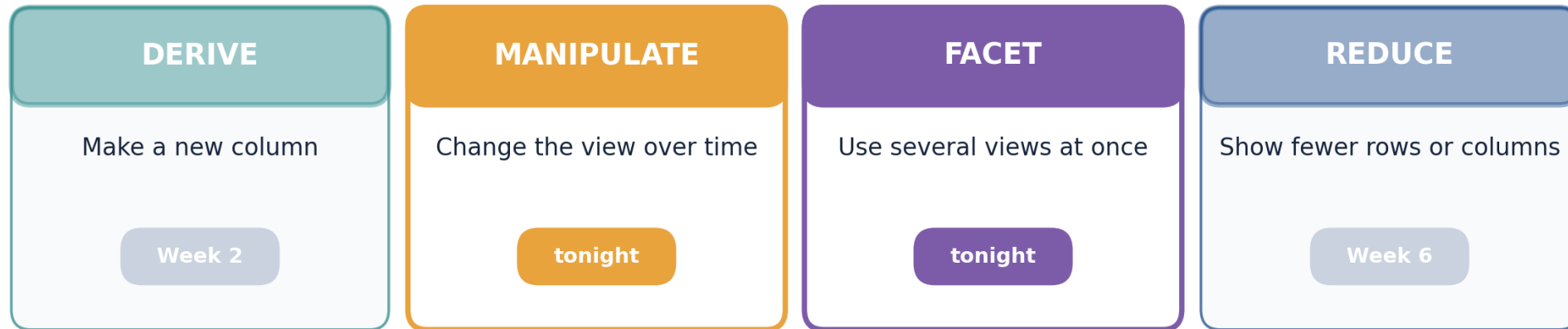
IN ONE LINE

Last heavy theory night. From Week 5 it is mostly tools and the project.

1	What's Vis, and Why Do It?	done
2	Task abstraction · Marks & channels	done
3	Arranging tables in space	tonight
4	Interaction · Multiple views	tonight
5	Map colour · Tableau 1	Assignment 3
6	Spatial data · Networks · Tableau 2	Project starts
7	Midterm · Project session 1	Midterm
8	Assignment review · Project session 2	
9	Tableau 3 · Project session 3	
10	Tableau 4 · Project session 4	
11	Final presentations · What's next	Project due

Two weeks in one deck. Week 3 is where things go on the page. Week 4 is what happens when the reader can touch them.

Four ways to cope when there is too much data



Four ways to cope when there is too much data for one simple picture. Tonight covers the middle two.

There is a limit to how much you can show in one simple picture. When you hit it, there are only four moves available, and every complicated visualization you have ever seen is some combination of them.

You already know the first. Deriving a new column was Week 2's big idea. Tonight covers the middle two: changing the view over time, and using several views at once. Reducing — showing fewer rows or columns — comes in Week 6.



03

WEEK THREE

Where does everything go on the page?

You have a table. You have decided what shape to draw. Now: where does each shape go? That single question generates almost every chart type you know.

Three kinds of data — tonight is about the first

TABLES are rows and columns. A spreadsheet. Most data you will ever meet.

NETWORKS are things connected to other things. Friends, flights, citations.

SPATIAL data has real positions baked in — maps, scans, terrain.

The difference matters because with a table you get to choose where things go. With a map you do not: the position is the data. Networks sit in between.

Tonight is tables. Networks and maps come in Week 6.

IN ONE LINE

With a table, position is yours to spend. That freedom is the whole design problem.

Dataset Types

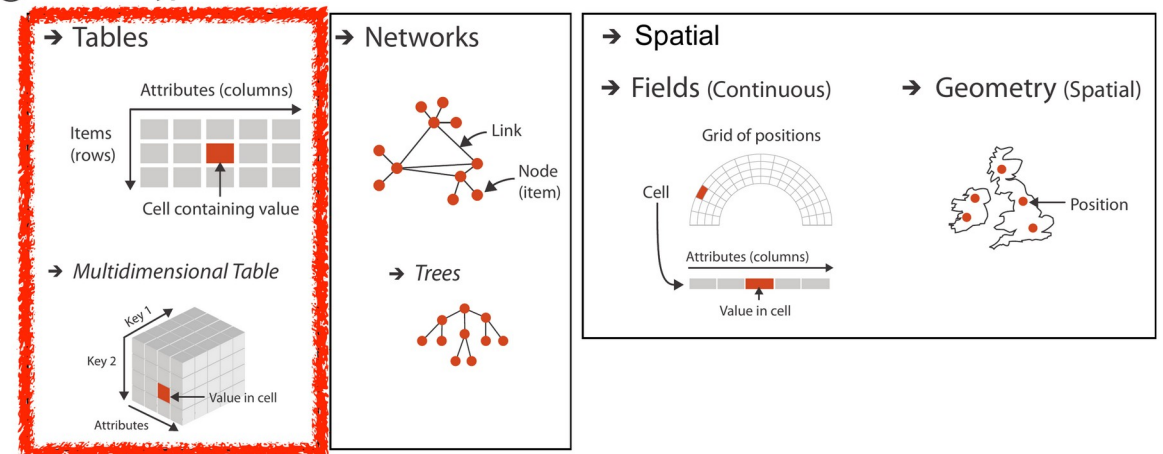


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Two kinds of column: keys and values

Before you can arrange anything, you have to know which columns identify a row and which ones measure it.

KEY

A column that tells you which row you are looking at. Country. Student ID. Month. Product name. Keys are independent — you do not compute them, they are given.

VALUE

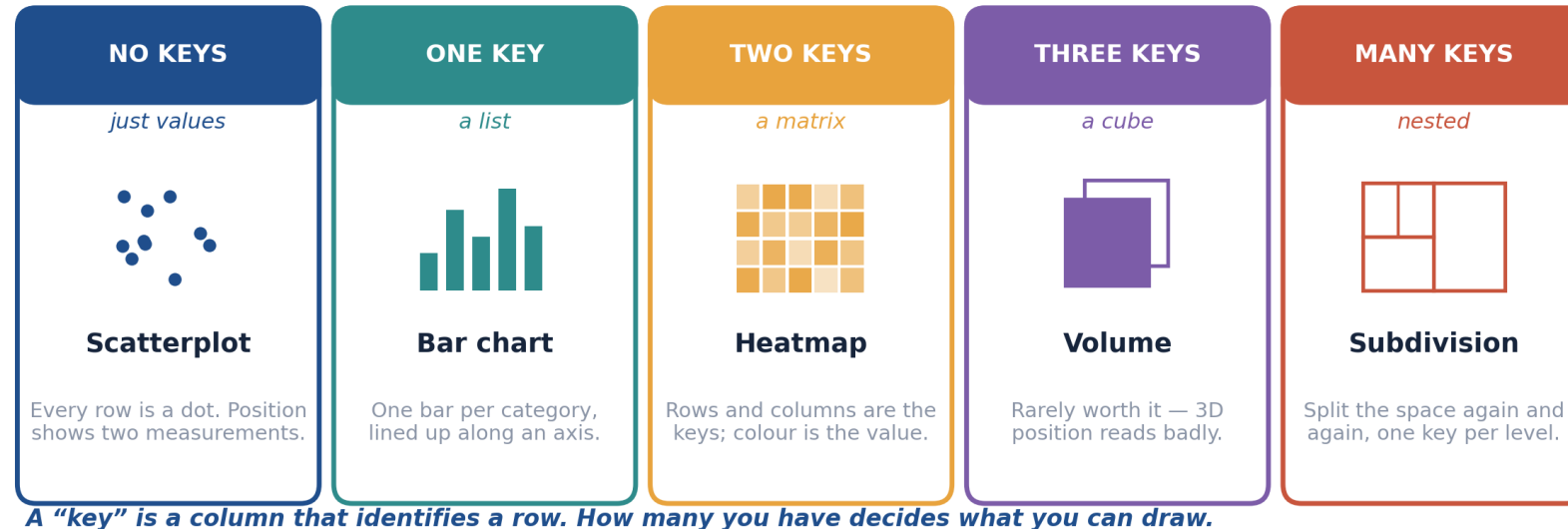
A column that measures something about that row. Population. Grade. Revenue. Temperature. Values are dependent — they depend on which row it is.

The test

Ask: if I change this column's value, am I now looking at a different row? If yes, it is a key. If no, it is a value.

In a spreadsheet of sales by region and month, region and month are keys; the sales figure is the value.

How many keys you have decides what you can draw



This is the single most useful page of Week 3. Count the key columns in your table, and the list of sensible charts shrinks immediately.

No keys at all means every row is just a pair of measurements, so you get a scatterplot. One key gives you a list, which lines up as a bar chart. Two keys give you a grid, which becomes a heatmap. Beyond that it gets hard fast.

No keys: the scatterplot

The simplest case. You have two measurements for each row and nothing identifying it.

Put one measurement on the horizontal axis and the other on the vertical. One dot per row.

This is the most information-efficient chart there is, because it spends both position channels — the two most accurate things you have — on actual data.

Notice the right-hand panel: the same data on a log scale. Changing the scale is part of arranging, not an afterthought.

IN ONE LINE

Two values, both on position. Nothing else comes close for accuracy.

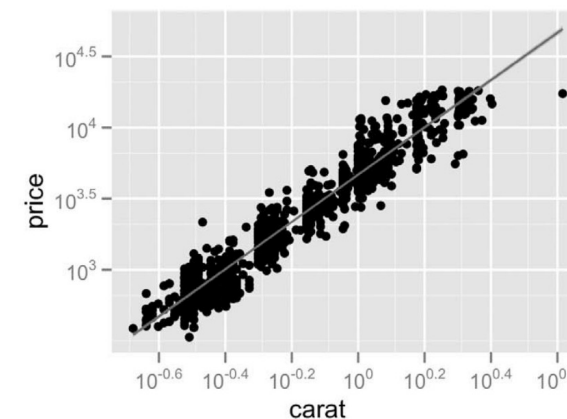
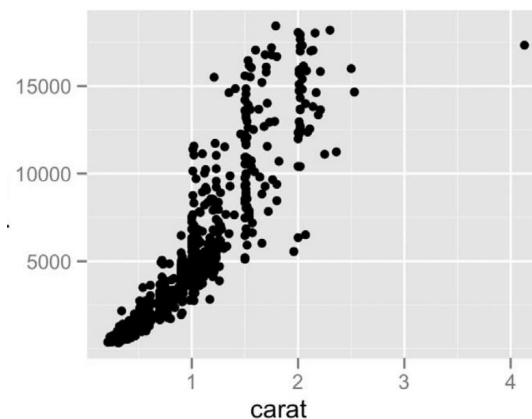
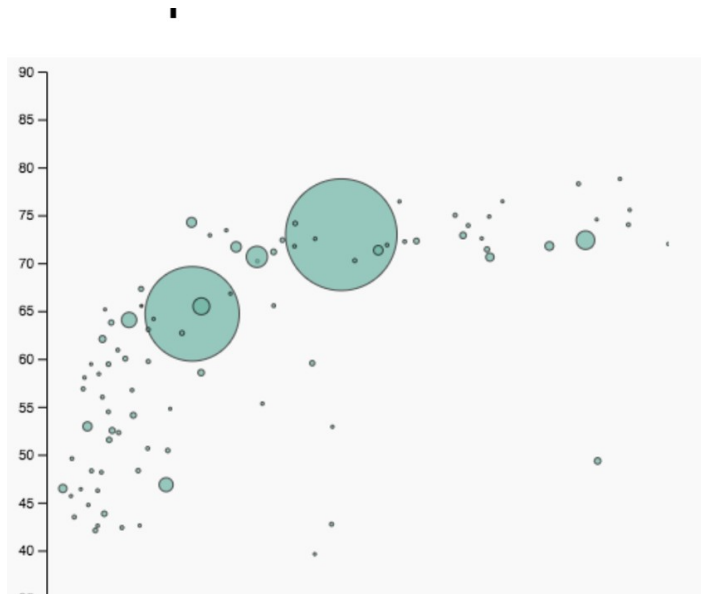
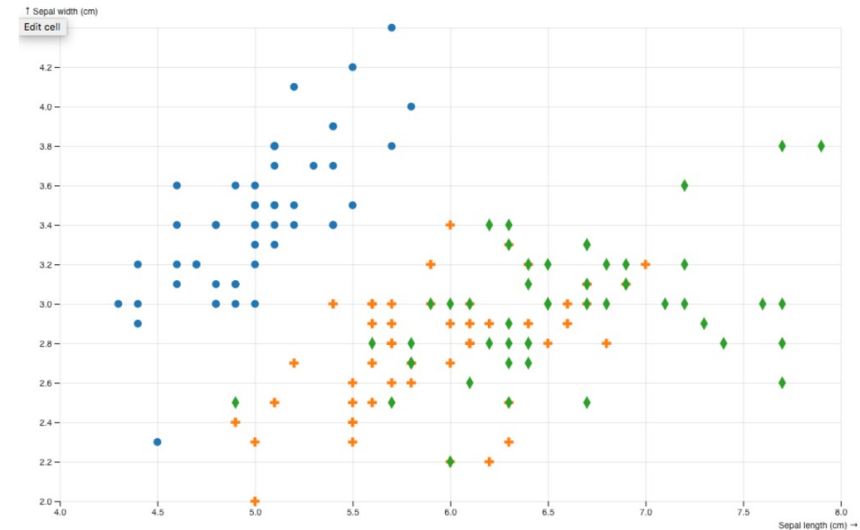


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Scatterplots can carry more than two things



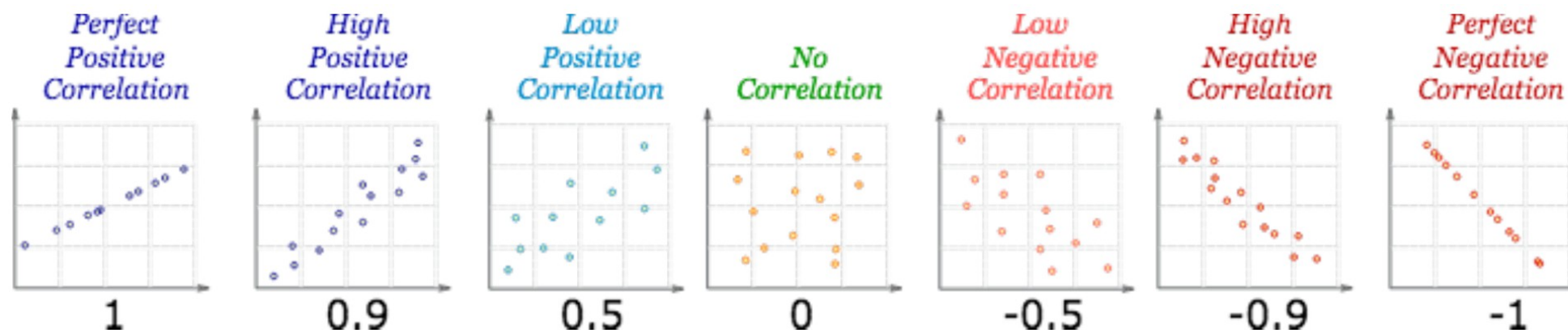
Once both positions are spent, you still have the weaker channels from Week 2 available. Size gives you a third value. Colour hue gives you a category. Shape gives you another category.



The warning from Week 2 still applies: those channels are read far less accurately than position, so put your least important attributes there. And remember the shape of a point mark has not spent anything yet — that is why scatterplots are so extensible.

Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

What scatterplots are actually good at



<https://www.mathsisfun.com/data/scatter-xy-plots.html>

Reading the strength and direction of a relationship. Left to right the correlation runs from perfect positive to perfect negative, and you can judge roughly where a real chart sits without computing anything.

Note how quickly it becomes hard. Telling 0.9 from 0.5 is easy. Telling 0.5 from 0.3 is not. So a scatterplot is excellent for 'is there a relationship and roughly how strong', and poor for 'exactly how strong'. If the exact number matters, print the number.

Correlation reference strip via mathisfun.com. Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

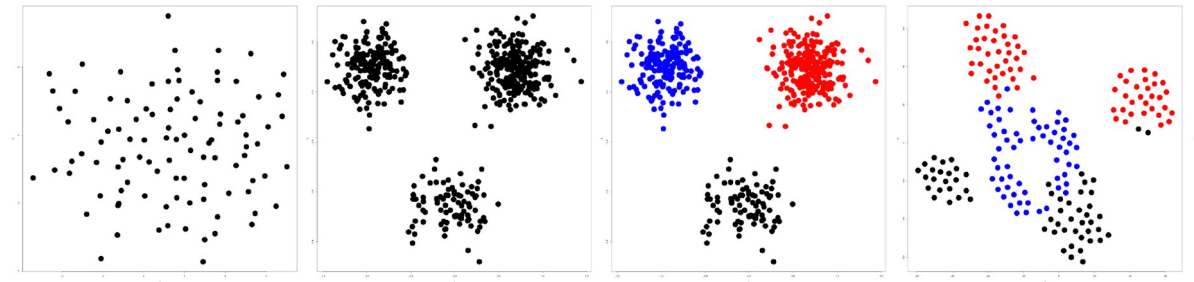
And at spotting groups

The other thing scatterplots do well: showing that the rows fall into clumps.

The top row shows clusters appearing in the raw positions — no colour needed, your eye finds them.

The bottom row adds colour for a known category, so you can check whether the groups you already know about line up with the clumps in the data.

That second case is one of the most common real uses of a scatterplot in practice.

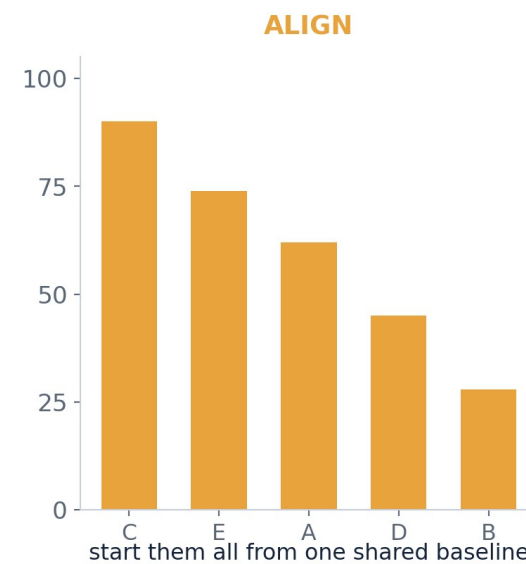
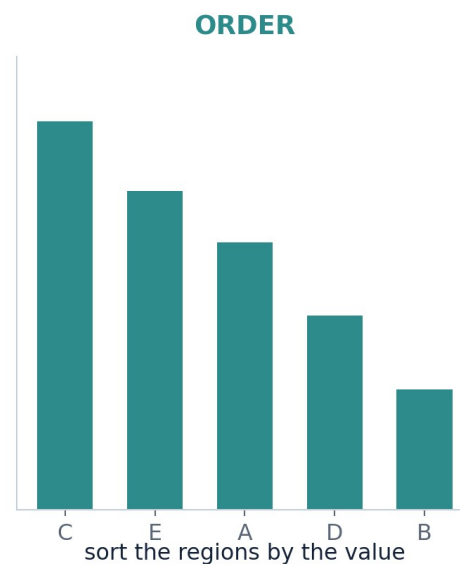
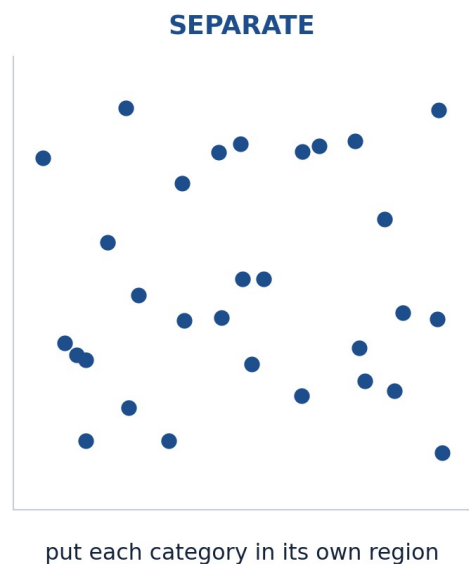


IN ONE LINE

Clumps in position, categories in colour. Comparing the two is the analysis.

Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

One key: three moves that build a bar chart



With one key column, you divide the page into regions — one per category. Three separate decisions follow, and they are usually made without thinking.

SEPARATE means each category gets its own region. ORDER means you choose the sequence those regions appear in. ALIGN means they all start from the same baseline. Do all three and you have a bar chart. Skip one and you have something noticeably worse, as the next two slides show.

The result: the bar chart

One line mark per category. Length carries the value, position along the axis carries the key.

Both charts show identical numbers. The left one is in alphabetical order; the right one is sorted by value.

The sorted version answers 'which is biggest' and 'what is the ranking' instantly. The alphabetical version makes you work for both.

Alphabetical order is almost never what the reader needs. It is simply the default your tool gave you.

IN ONE LINE

Sorting a bar chart costs one click and is almost always worth it.

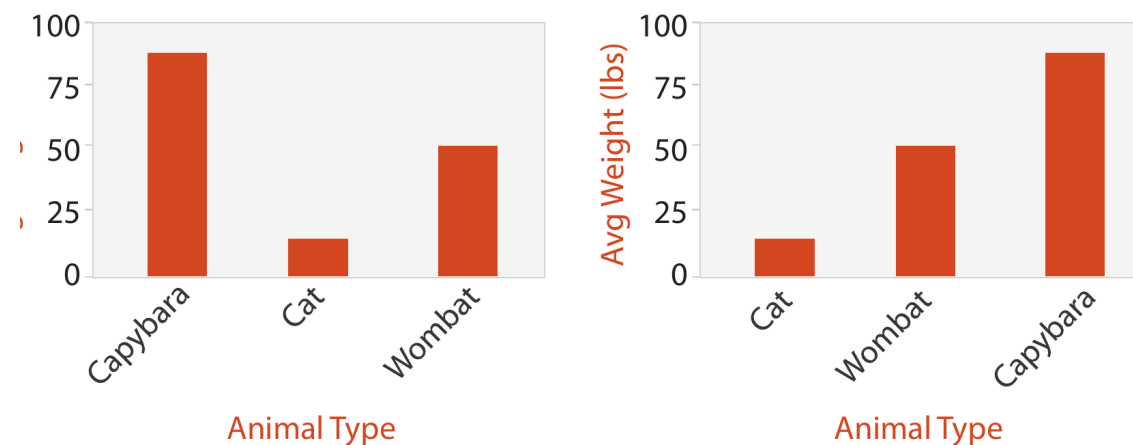
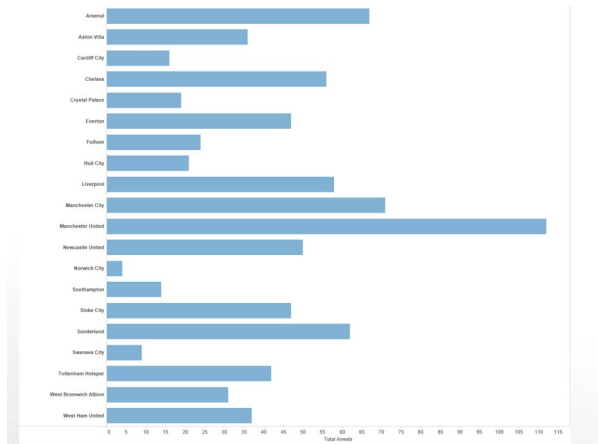


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

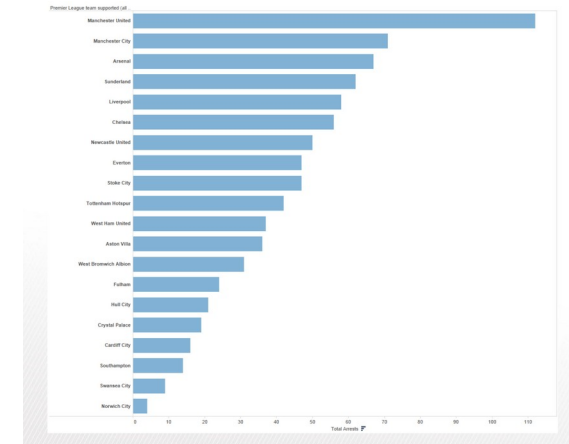
What happens when you skip the ordering

Same clubs, same numbers, same chart type. Only the order changed.



Alphabetical

Try answering 'which club is fourth highest'. You have to scan every bar, hold candidates in your head, and compare them. With this many categories that is genuinely hard, and most people get it wrong.



Sorted by value

Now the same question takes one glance — count four down from the top. Sorting converted a search task into a lookup task, which is exactly the Week 2 vocabulary for why it helps.

Football banning orders by club, Ben Jones / UK Home Office. Munzner, Visualization Analysis and Design, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

And when you skip the aligning

Here the categories are separated — each one has its own region — but nothing is ordered and nothing shares a baseline.

The value is encoded as circle area, and the circles are packed to fill the space.

It looks appealing. It is nearly useless for comparison. You are judging areas with no common reference, which is the worst case from Week 2's effectiveness ranking.

The honest assessment: this is a decorative layout, not an analytical one. Which is fine if decoration is the job.

IN ONE LINE

Pretty is not the same as readable. Know which one you are buying.



Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Stacked bars: more than one value per category

Sometimes each category has several values that add up to a meaningful total — sales split by region, votes split by party.

Stacking puts them end to end in one bar. The total is the full bar height, and each segment is one part.

This works because the parts genuinely sum to the whole. Stacking things that do not add up to anything is a common and confusing error.

But stacking has a cost, and it is the Week 2 baseline problem again — only the bottom segment starts at zero.

IN ONE LINE

Stack only when the parts really do add up to the total.

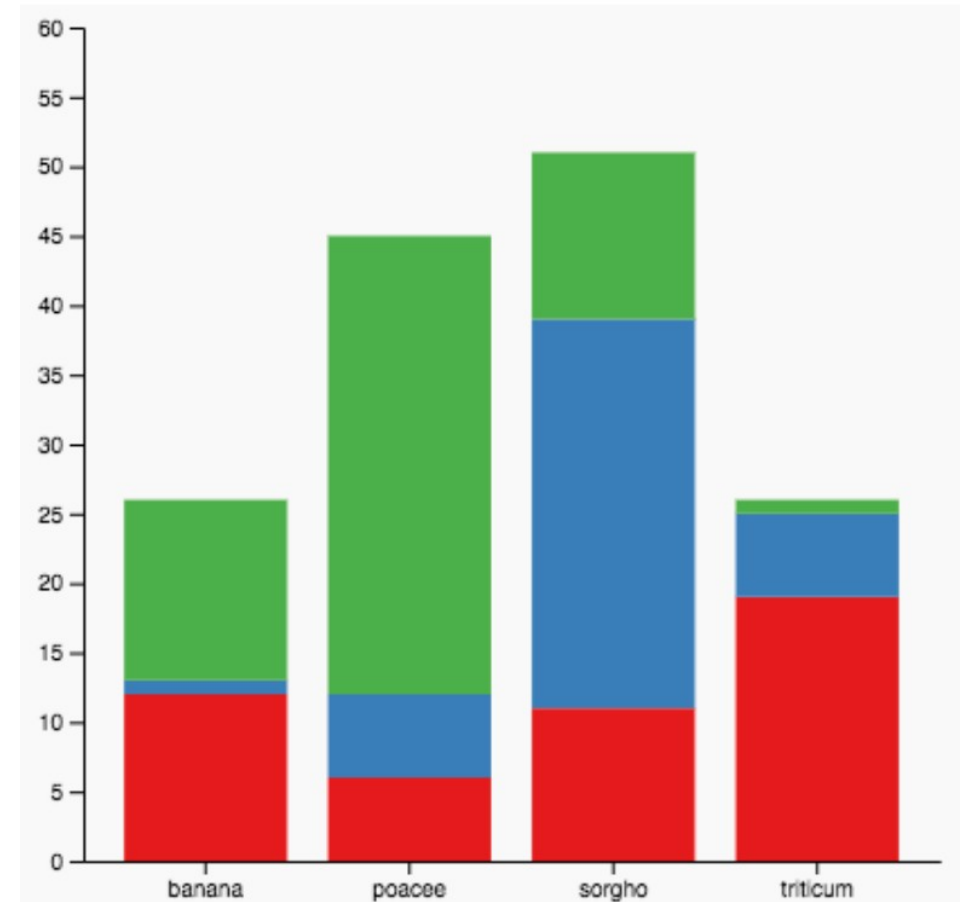
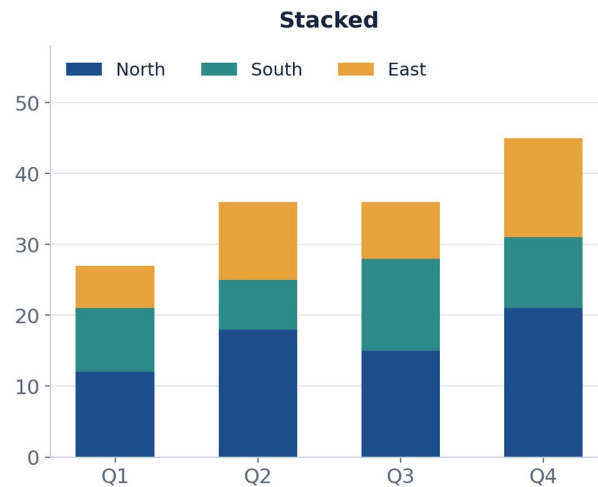
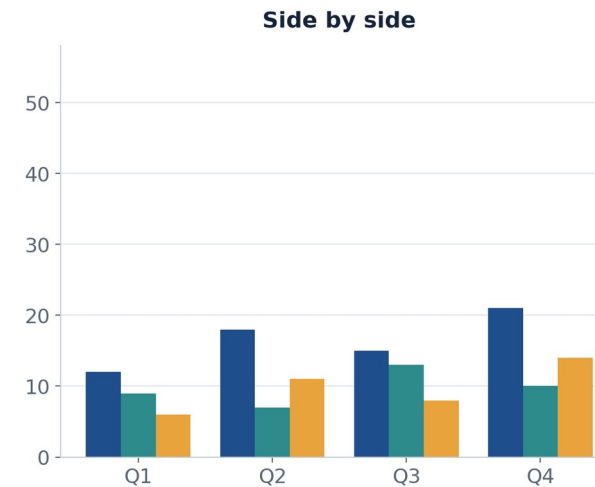


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Stacked or side by side? It depends on the question



Totals are easy. Only the bottom colour sits on a shared baseline, so the others are hard to compare.



Every bar starts at zero, so comparing within a quarter is easy. Totals now have to be done in your head.

These two charts contain exactly the same twelve numbers. Neither is better in the abstract — they support different tasks, which is the Week 2 lesson all over again.

If the reader needs the total for each quarter, stack. If the reader needs to compare regions within a quarter, go side by side. If they need both, you probably need two charts — which is Part 2's whole subject.

Streamgraphs: stacked bars that flow

A streamgraph is a stacked area chart with the baseline moved to the middle instead of the bottom.

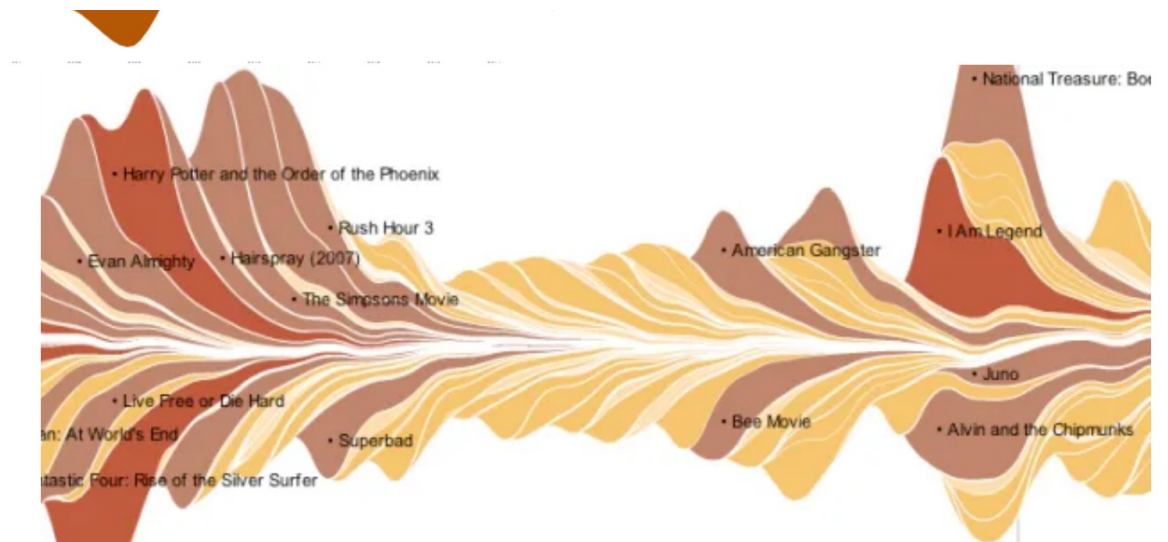
It is good at one thing: showing how the overall shape of a whole changes over time, and how individual parts swell and shrink inside it.

It is bad at almost everything else. No segment has a fixed baseline, so reading any individual value is guesswork.

Use it for an evocative overview of many parts over time. Do not use it when anyone needs a number.

IN ONE LINE

Beautiful for shape over time. Hopeless for reading values.



Box office receipts, flowingdata.com. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Dots and lines: when the key has an order

If the key column has a natural sequence — years, months, grades — you have another option.

Use point marks positioned by value instead of bars, and you get a dot plot. Connect the dots and you get a line chart.

Dropping the bars frees you from the zero baseline, which means you can zoom in on the interesting range without lying.

The connecting line adds a claim: these points are part of one continuous thing. Make sure that claim is true.

IN ONE LINE

**Lines encode change as slope. Bars encode amount as length.
Different jobs.**

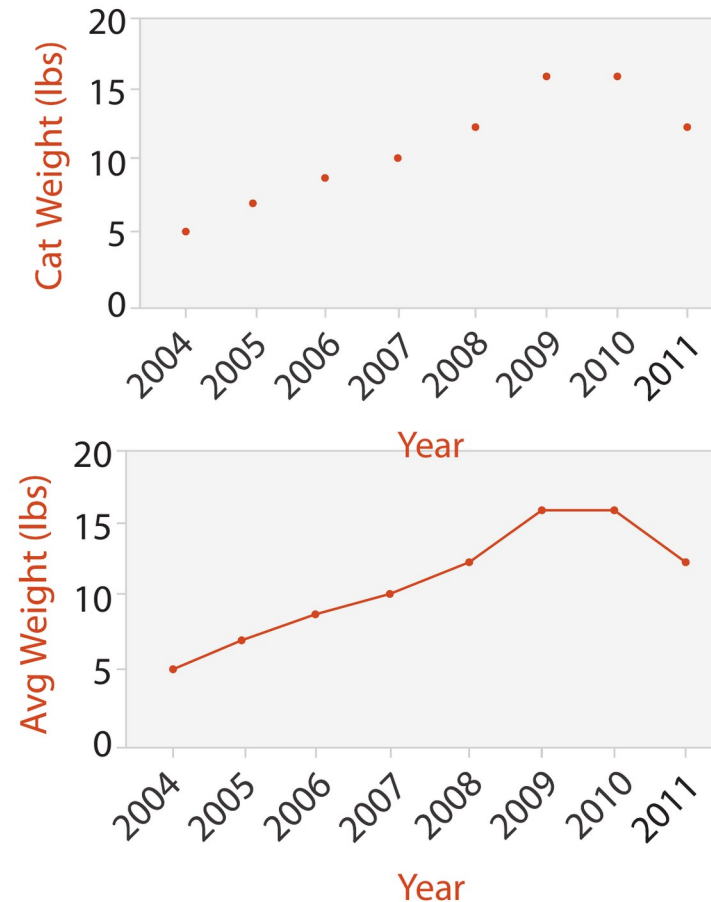


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Bars or lines? The rule, and the evidence



The rule is simple. A line asserts that the points between your data points mean something. That is true for time and for any ordered scale. It is false for unordered categories — there is nothing between Female and Male to interpolate.

This is not just style. Zacks and Tversky tested it: people shown bars described discrete comparisons, and people shown the identical data as lines described trends. The chart type changed what viewers concluded, regardless of the numbers.

Study: Zacks & Tversky, Bars and lines, Memory & Cognition 27(6), 1999. Munzner, Visualization Analysis and Design, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Where you cut the axis changes the story

Both panels plot the same two Planned Parenthood services over the same years.

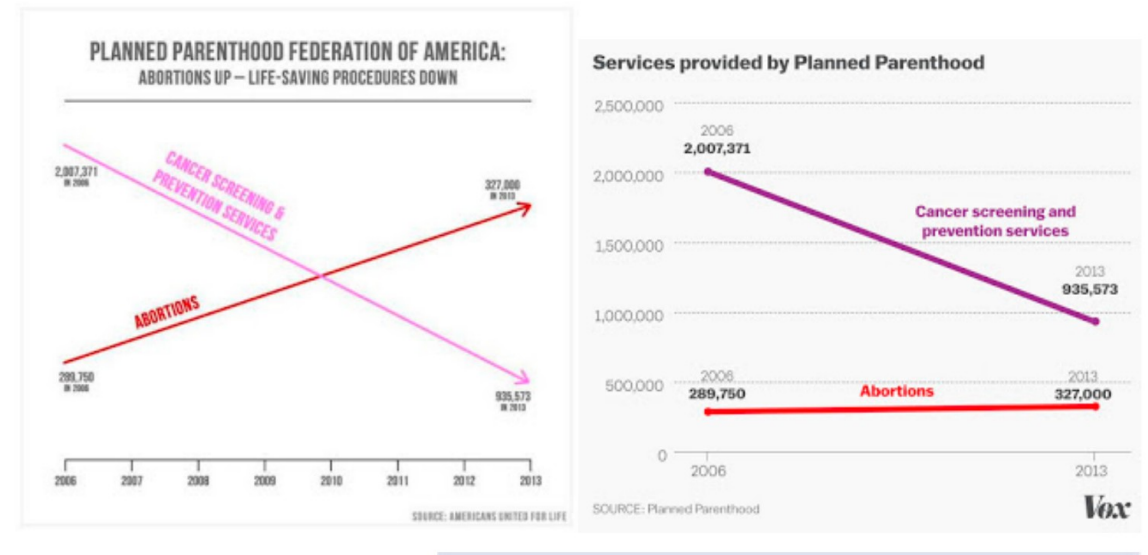
The left one was shown to Congress. It has no y-axis at all, the two lines are scaled differently, and the crossing point is pure theatre.

The right one plots both series on one honest axis. The lines still cross, but the magnitudes are now comparable and the story is far less dramatic.

This is the Week 1 truncated-axis lesson, one level more sophisticated — here the problem is not just the range, it is that two series share a picture without sharing a scale.

IN ONE LINE

Two lines in one frame imply one scale. If they are not on one, say so loudly.



Via Vox / Americans United for Life. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Dual axes: usually a bad idea

Two series with different units, plotted in one frame with a separate axis on each side.

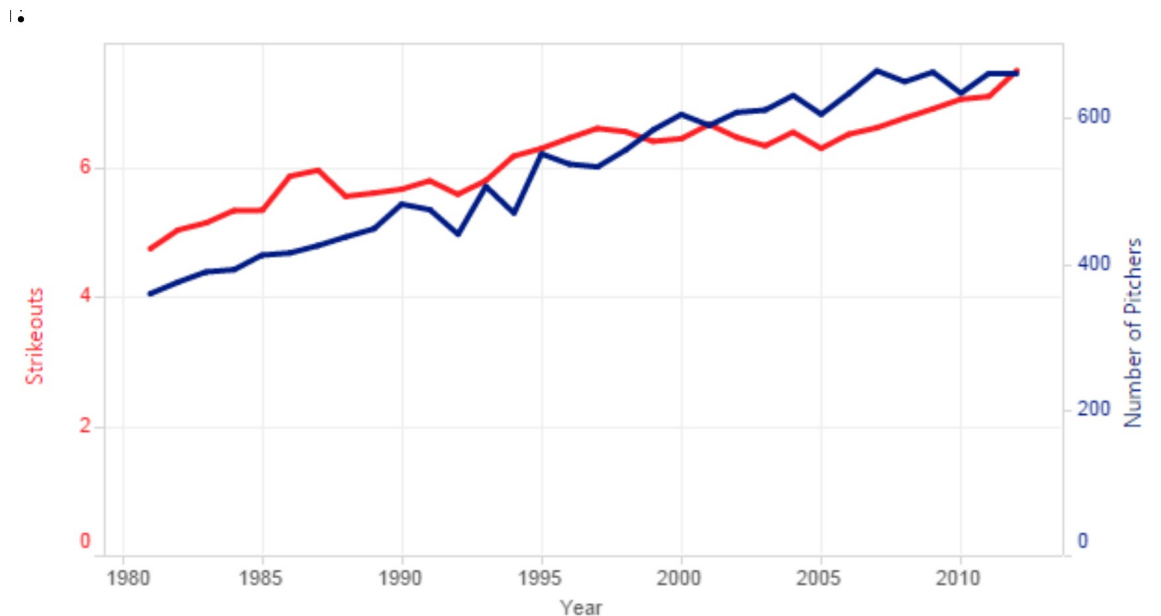
The problem is that the relationship between the two lines is entirely controlled by how you scale each axis. Slide one scale and you can make them diverge, converge or cross wherever you like.

So any apparent correlation is an artefact of your scaling choices, not a property of the data.

Better options: index both series to a common starting point, plot them in two stacked panels sharing an x-axis, or derive the ratio and plot that.

IN ONE LINE

If you can move the story by moving an axis, it was never the data's story.



Source | <http://www.baseball-reference.com/leagues/MLB/pitch.shtml> Ben Jones (@DataRemixed) | 5/4/2013

Ben Jones, baseball-reference.com data. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, *dataviscourse.net*.

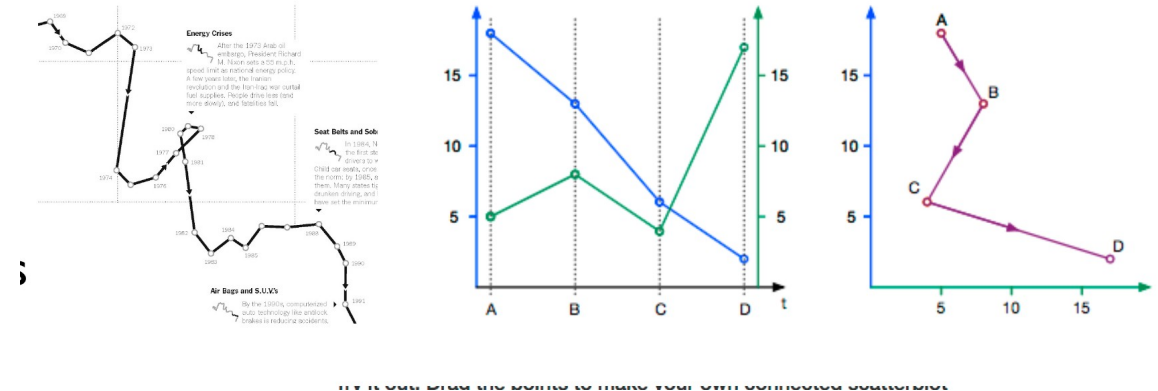
Connected scatterplots: time without a time axis

An unusual arrangement. Both axes carry values, and the line connects the points in time order.

So the path traces how two quantities moved together — loops mean cycles, long straight runs mean a sustained trend.

They are striking and they pack a lot in. They are also genuinely hard to read: viewers often miss that the line encodes time rather than connection.

Label the start, label the end, and label a few points along the way. Without that, nobody can orient themselves.

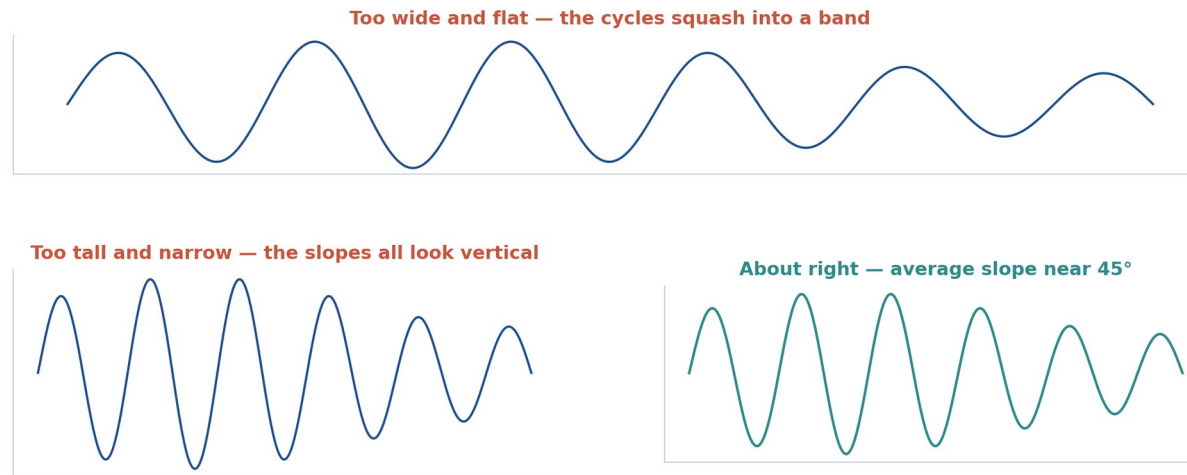


IN ONE LINE

Powerful, but only with heavy labelling. Never assume the reader will work it out.

Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

The shape of the box changes what you see



Same numbers in all three. Only the shape of the box changed.

Three panels, identical numbers. Only the width-to-height ratio differs, and the apparent behaviour of the data changes completely.

Squash it flat and the cycles disappear into a band. Stretch it tall and every slope looks vertical, so you cannot tell steep changes from gentle ones. The guidance from Cleveland is to aim for an average line slope near 45 degrees — that is where the eye compares slopes most accurately.

Banking to 45 degrees, after Cleveland; Munzner VAD Ch. 7.

Indexed lines: comparing things of different sizes

A recurring problem: several series you want to compare, but they are on wildly different scales. A big one squashes all the others flat.

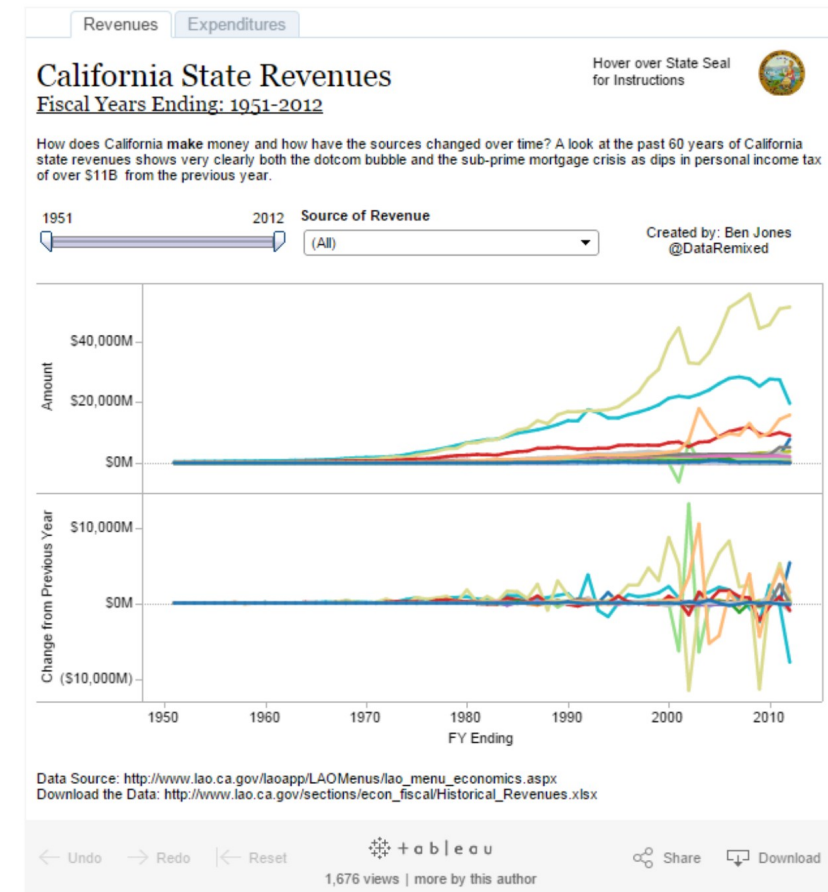
The fix is to derive new values. Divide every series by its own starting value, so all of them begin at the same place.

Now the chart shows relative change rather than absolute level, and every series is comparable.

This is the Week 2 derive idea applied to a layout problem. It is also the honest alternative to a dual axis.

IN ONE LINE

When scales differ wildly, index them to a common start instead of adding a second axis.



California state revenues, Ben Jones / Tableau Public. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Gantt charts: when the value is a span, not a point

Sometimes each row has a start and an end rather than a single number — a task, a reign, a contract, a hospital stay.

Draw a line mark whose position carries the start and whose length carries the duration.

One key down the side, time across the bottom. Simple, and surprisingly rare outside project management.

The limitation: with many overlapping rows it becomes dense fast, and ordering the rows well matters enormously.

IN ONE LINE

Two numbers per row — a start and a length — and the bar chart handles both.

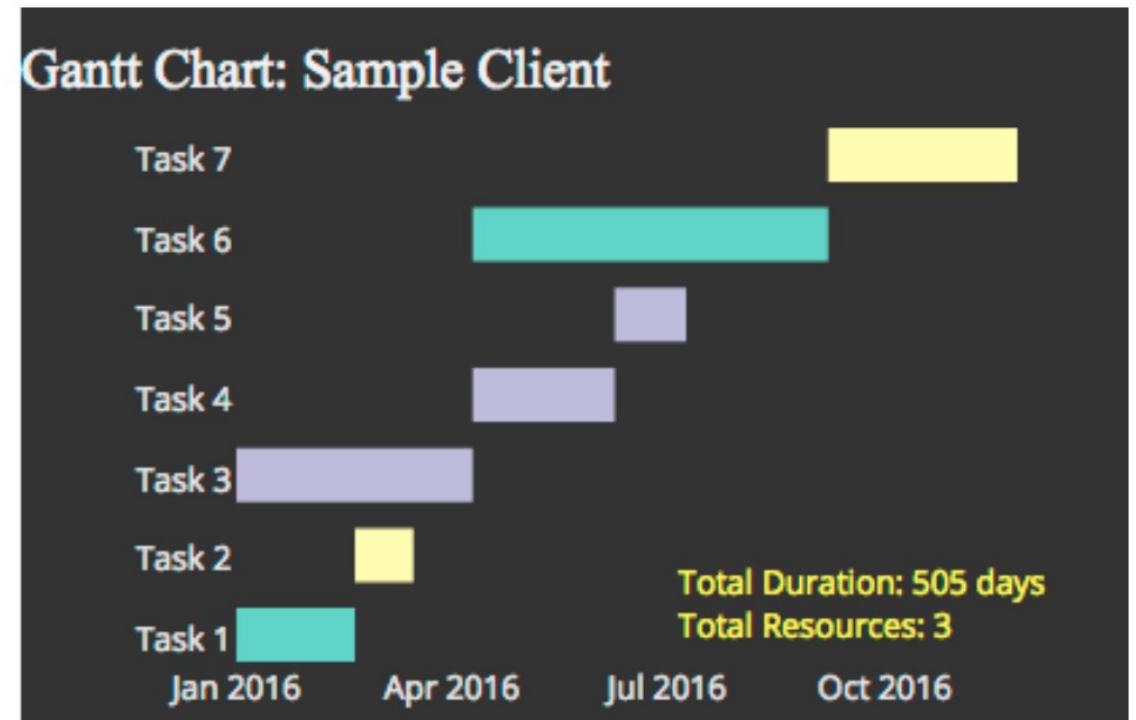


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Slopegraphs: exactly two time points

You met one of these in Week 1, with the triglyceride exercise.

Two vertical axes, one per time point. Each row is a line between them.

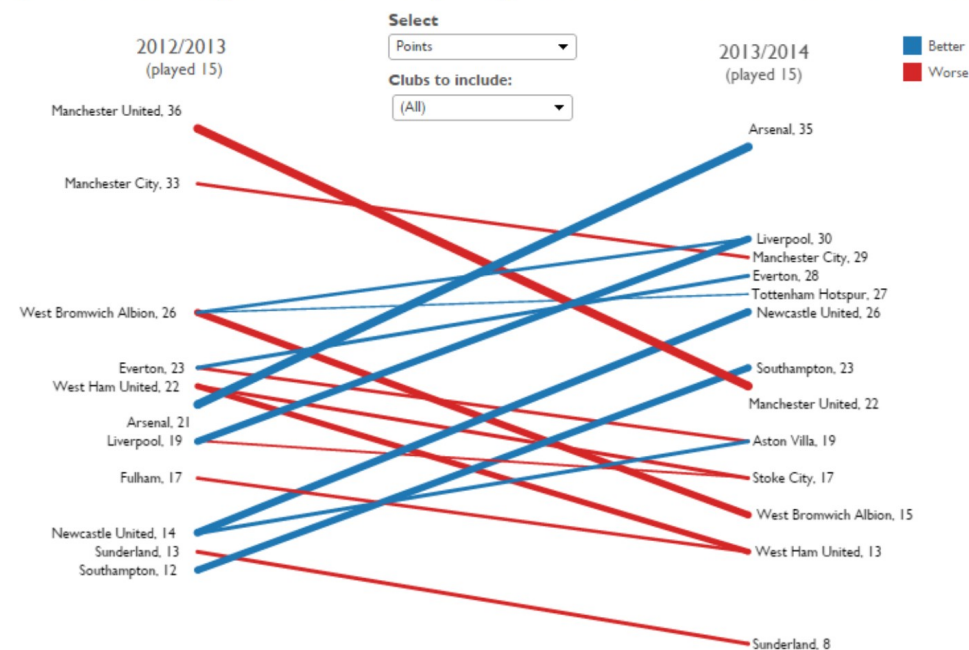
Because change is encoded as slope, a line going the wrong way is impossible to miss — and that is usually the whole point of the chart.

It only works for two time points. Three or more and you are back to an ordinary line chart.

IN ONE LINE

Two moments, many items, and the exceptions announce themselves.

irclay's Premier League Tables: Comparing 2012/2013 Starts to 2013/2014 Starts



Data:



Ref:



By:



Premier League tables, Ben Jones / Tableau Public. Munzner, Visualization Analysis and Design, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Breaking the rules on purpose

A chart of Edgar Allan Poe's publications. The bars hang downwards from the top, the palette is blood red, and the whole thing is deliberately gothic.

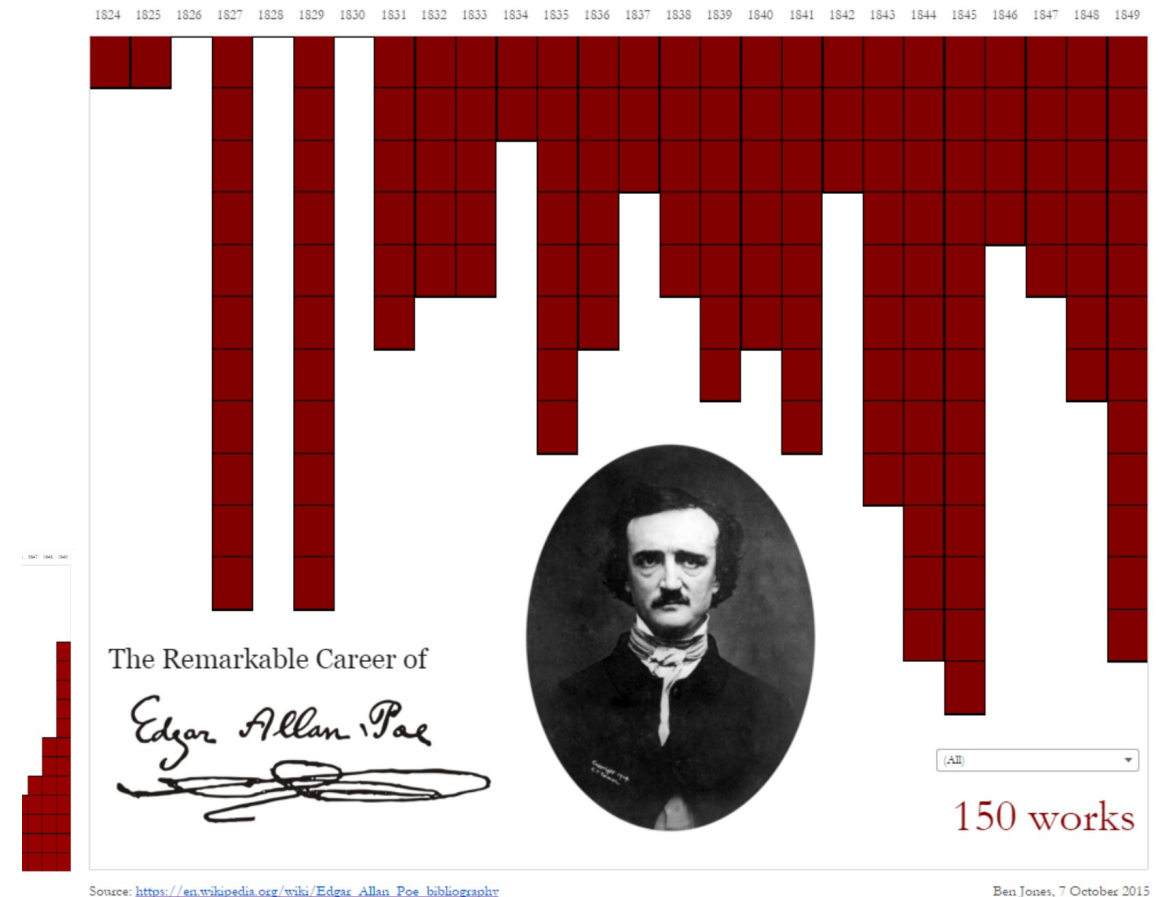
Every convention we have discussed is broken. It also works.

The difference between this and the bad examples earlier is intent and consistency. The designer knew the rule, chose to break it for effect, and broke it consistently so the reader could still decode the chart.

Breaking a rule you do not know about is just a mistake. Breaking one deliberately, in service of the piece, is design.

IN ONE LINE

Learn the conventions first. Then you have earned the right to break them.



Ben Jones, Tableau Public. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Two keys: the heatmap

With two key columns, every combination of the two is a cell in a grid. Rows are one key, columns are the other.

Position is now entirely spent on the keys, so the value has to go somewhere else — and that leaves colour.

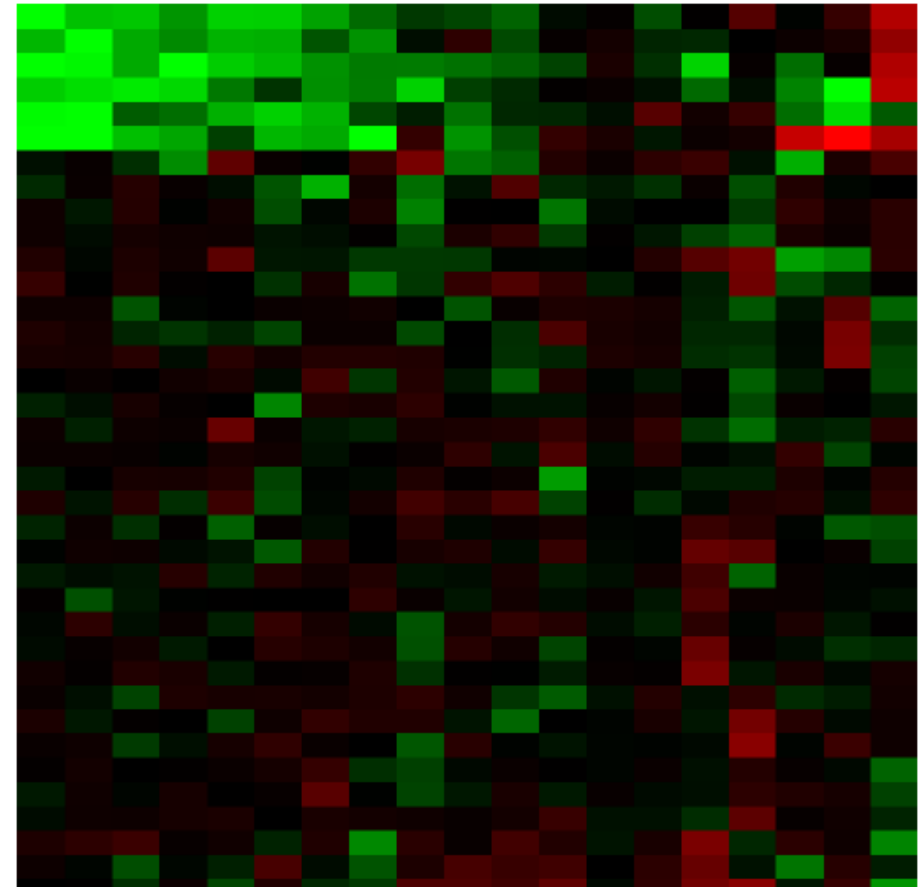
That is the heatmap's bargain. You can show an enormous grid at a glance, but you are reading values through a weak channel, so precision is gone.

Heatmaps are for spotting blocks, stripes and gaps. They are not for reading numbers.

IN ONE LINE

Two keys eat both positions. Colour is what is left, so expect to lose precision.

Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.



Cluster heatmaps: let the data choose the order

A heatmap's rows and columns have no natural order, and the order you pick decides whether any pattern is visible at all.

So compute one. Run a clustering algorithm over the rows and over the columns, reorder both by the result, and draw the trees on the edges.

Blocks of similar colour now appear because similar rows have been placed next to each other — and the trees tell you how the grouping was arrived at.

This is the standard display in genomics, and the same trick works for any wide table.

IN ONE LINE

Reordering is a design decision. Sometimes the best order has to be computed.

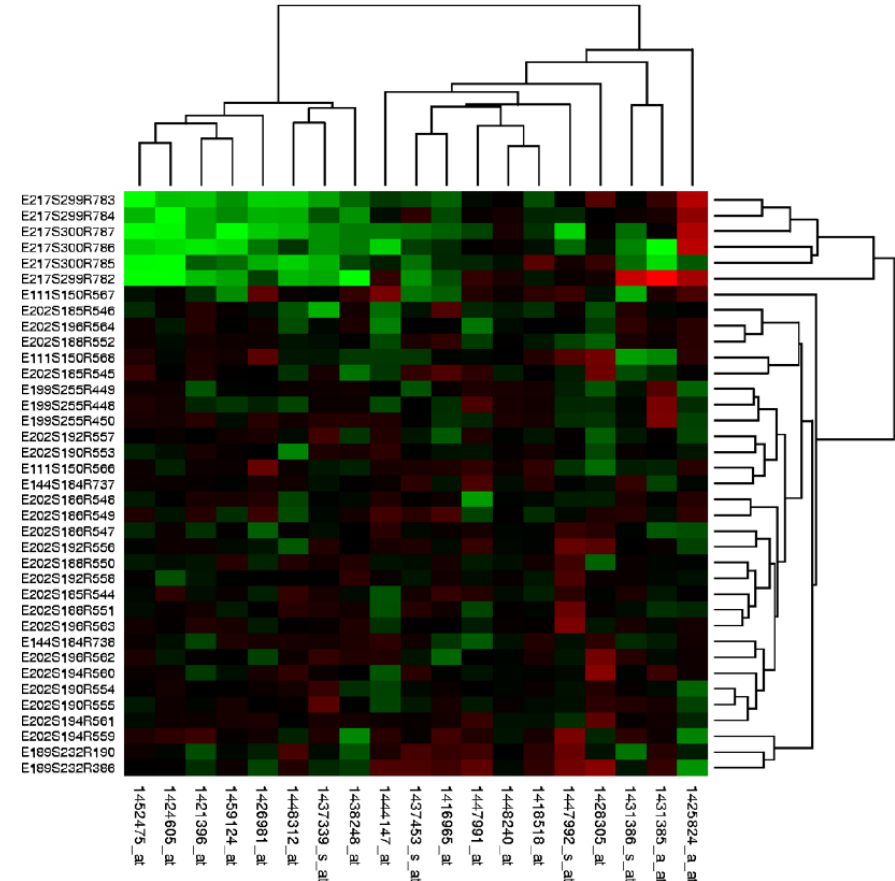
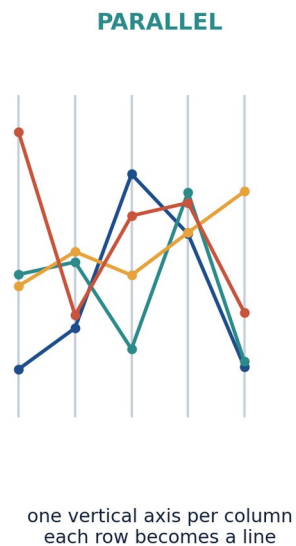
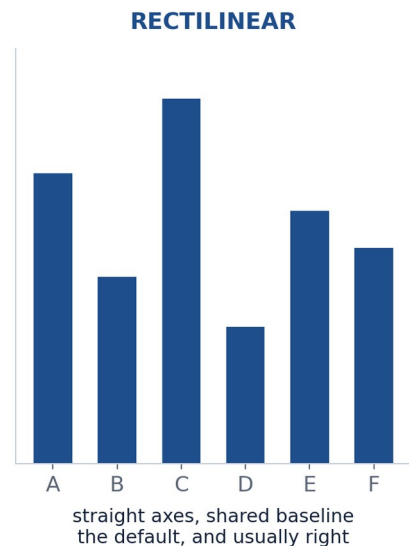


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

You can bend the axes — but it costs you



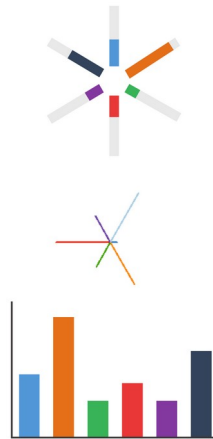
So far every arrangement has used straight axes meeting at a corner. That is rectilinear, and it is the default for good reason.

Parallel puts one vertical axis per column, side by side, and draws each row as a line crossing all of them. Radial fans the axes out from a centre. Both let you fit more columns on the page — and both make comparison harder, because position on a curve or on an unaligned axis is read far less accurately than position on a shared straight scale.

After Munzner VAD Ch. 7.

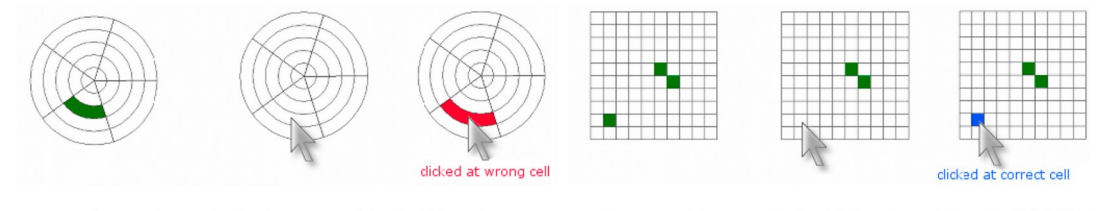
The same bar chart, wrapped into a circle

Three versions of one small table: radial bars, a star plot, and the ordinary straight version.



Radial bar, star, and plain bar

Top: bars radiating from a centre. Middle: the same values as a star, with the tips joined. Bottom: the plain bar chart. All three encode identical numbers, and the bottom one is the easiest to read.



And the measured cost

The study on the right compared pointing accuracy in radial versus rectangular layouts. People made more errors, and slower ones, in the radial condition — and the errors cluster where the cells are most distorted.

Right: Diehl, Beck & Burch, *Uncovering strengths and weaknesses of radial visualizations*, IEEE TVCG 2010. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Radar plots: and why we met them in Week 2

A radar plot puts each column on its own spoke and joins the values into a closed shape.

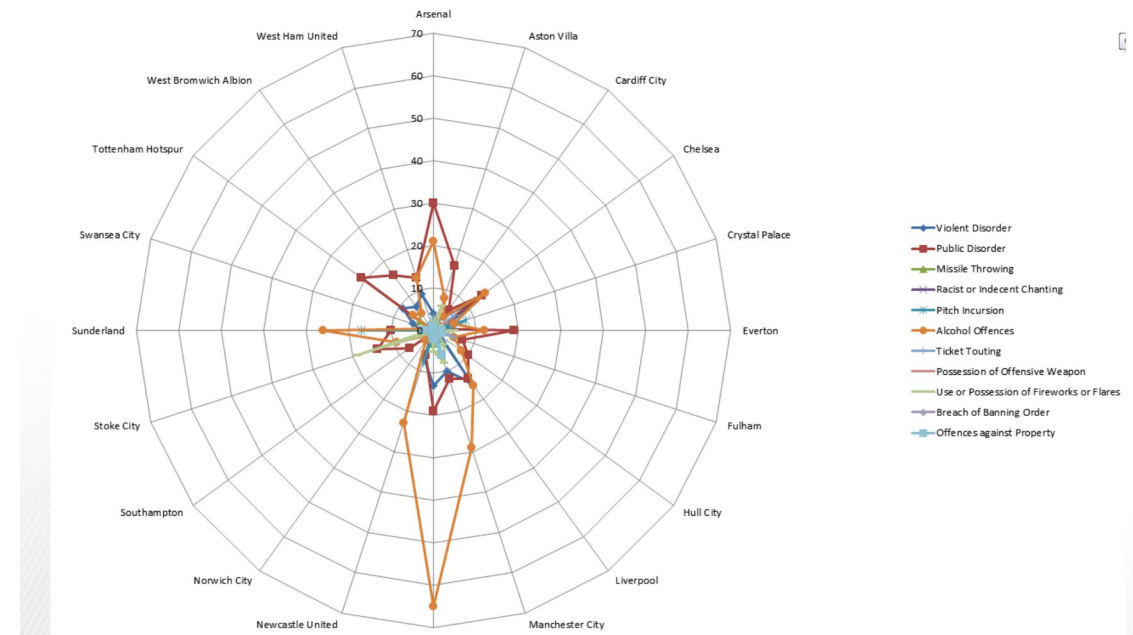
You met these in the Week 2 exercise, where you scored one against a bar chart for six different tasks.

The finding then still holds. Comparing values on neighbouring spokes is easy; comparing values on opposite spokes is hard, and the order of the spokes is an arbitrary design choice that decides which comparisons are easy.

They also break down when the categories have no cyclic meaning — which is most of the time.

IN ONE LINE

Good for overall shape and for memorability. Bad for ranking anything.



Football club profiles. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

The famous one: Nightingale's rose

Florence Nightingale's 1858 diagram of mortality in the Crimean War. Each wedge is a month; the area shows deaths, split by cause.

It is radial, which by everything we just said should make it hard to read.

But the data is genuinely periodic — twelve months wrapping round a year — and the point being made is a comparison of two whole years, not a reading of individual values.

It also worked. It is credited with helping persuade the British government to reform army sanitation.

IN ONE LINE

The exception that shows the rule: radial earns its place when the data really does cycle.

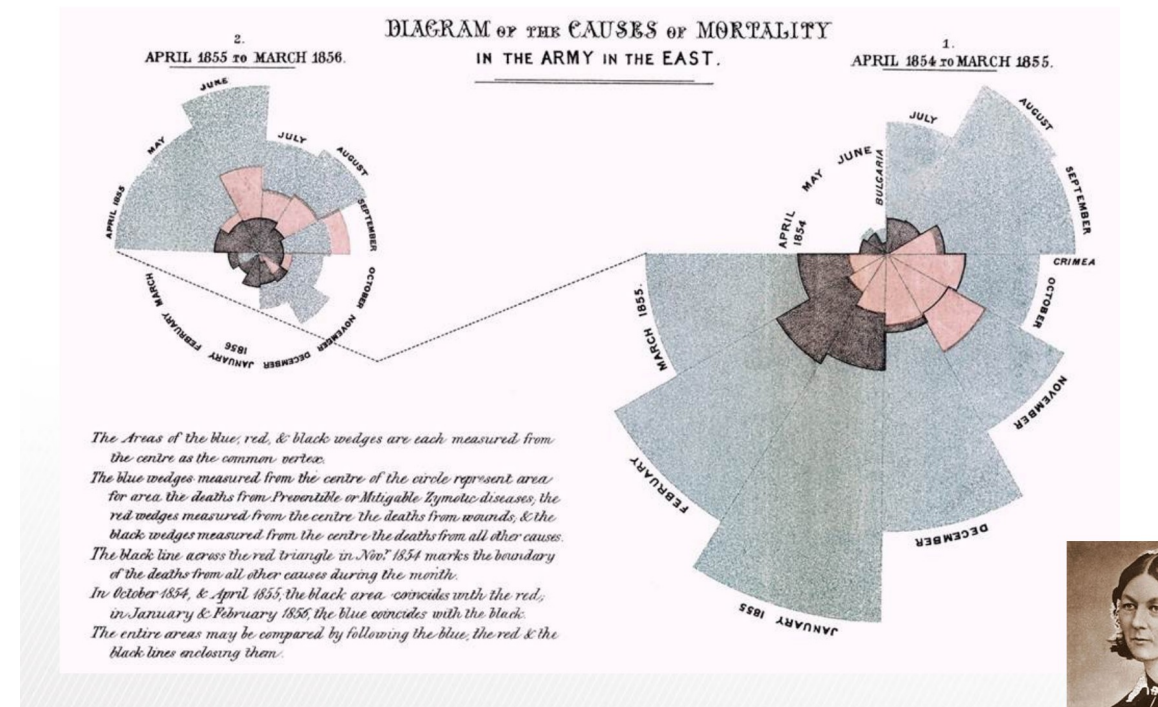
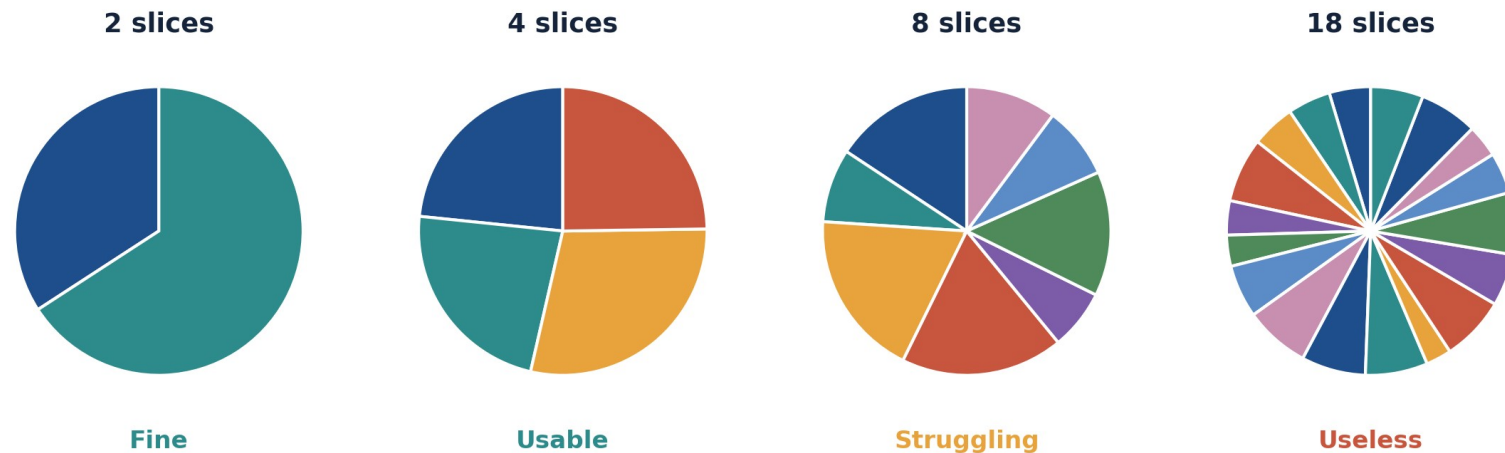


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Pie charts: what they can and cannot carry



A pie chart can carry two or three parts of a whole. Past about five, nobody can rank the slices.

A pie chart is a radial layout with one key and one value, where the value is encoded as angle and the parts must sum to a whole.

With two or three slices it is perfectly readable, and the part-to-whole relationship is immediate in a way a bar chart does not match. As the slice count climbs, the task shifts from 'is this about half' to 'rank these', and angle is simply not accurate enough for ranking. By eighteen slices there is nothing a reader can do with it.

After Munzner VAD Ch. 7.

And the measured version of that claim

A study comparing how accurately people read several part-to-whole encodings against a standard pie chart.

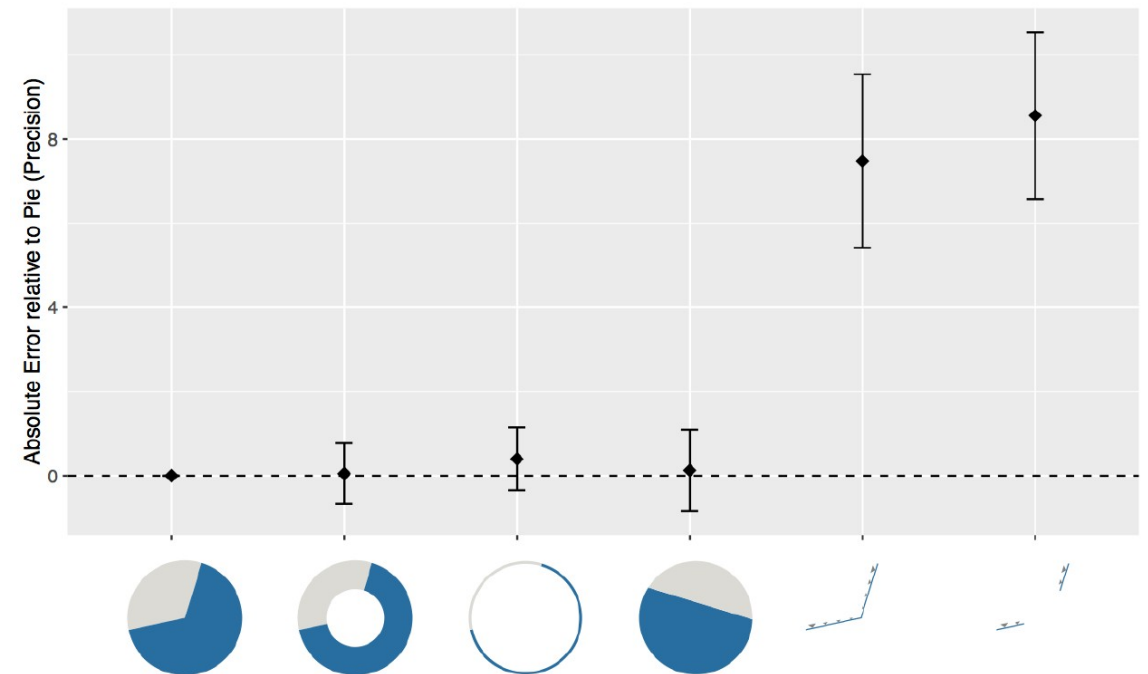
Read the vertical axis as error relative to the pie: at zero, an encoding is as accurate as a pie; above zero it is worse.

The result is more interesting than the usual slogan. Pies and donuts perform about the same as each other, and both are beaten by the stacked bar — but not by as much as the anti-pie orthodoxy suggests.

So the honest summary is not 'pies are evil'. It is 'pies are mediocre, and there is usually something better'.

IN ONE LINE

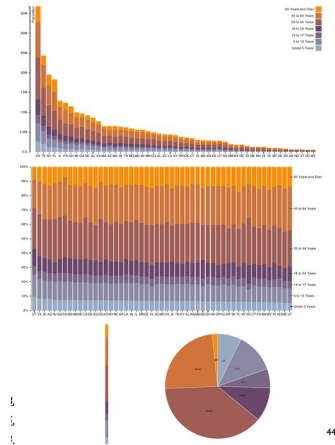
Follow the evidence, not the slogan. Mediocre is not the same as useless.



Skau & Kosara, Arcs, angles or areas, Computer Graphics Forum 2010/2016. Munzner, Visualization Analysis and Design, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

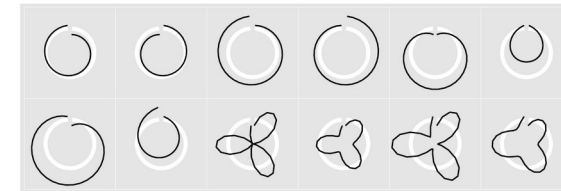
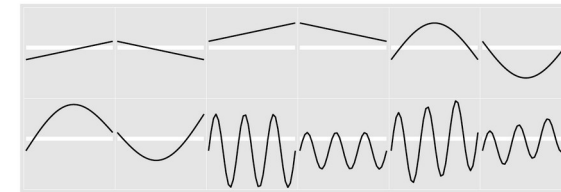
Two more part-to-whole options

When you have many categories and still need proportions.



Normalized stacked bars

Stack the bars and then stretch every one to the same height. Now each bar shows proportions rather than totals, and you can compare the mix across many categories at once — which a row of pie charts cannot do.



Glyphs: tiny charts as marks

A glyph is a small chart used as a single mark. Each one has internal structure, so you can place dozens across a layout and still read a little detail in each. The cost is that each one is small, so only coarse patterns survive.

Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Many columns: show every pair at once

A scatterplot matrix — a SPLOM. One small scatterplot for every pair of columns, arranged in a grid.

You get every pairwise relationship in one view, which is a genuinely efficient way to survey an unfamiliar table.

The cost is quadratic. Ten columns means forty-five plots; twenty columns means a hundred and ninety, each one tiny.

So it is a reconnaissance tool. You scan it, spot two or three pairs worth investigating, then draw those properly at full size.

IN ONE LINE

Use it to decide what to look at, not to do the looking.

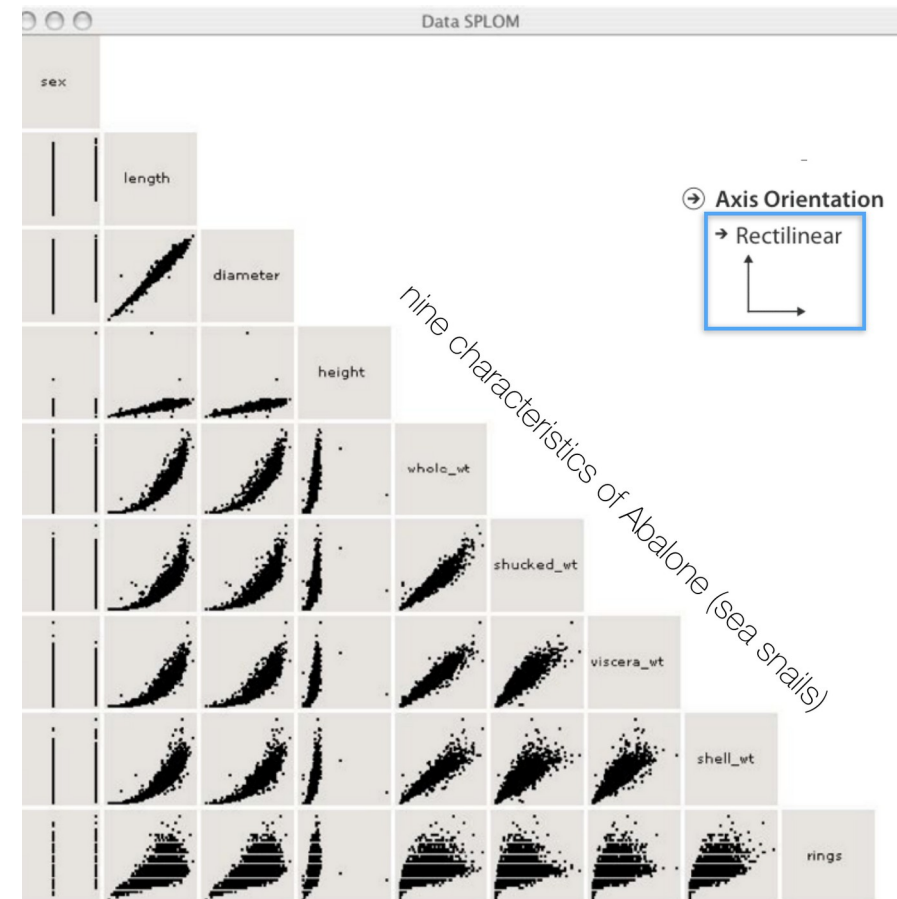


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

Or give every column its own vertical axis

Parallel coordinates. Each column gets a vertical axis, the axes sit side by side, and each row becomes a line crossing all of them.

Unlike a SPLOM, this grows linearly — twice the columns means twice the width, not four times the plots.

And because every row is one continuous line, you can follow an individual item across all its attributes, which a SPLOM cannot do.

The price is that only neighbouring axes can really be compared, and the axis order is yours to choose.

IN ONE LINE

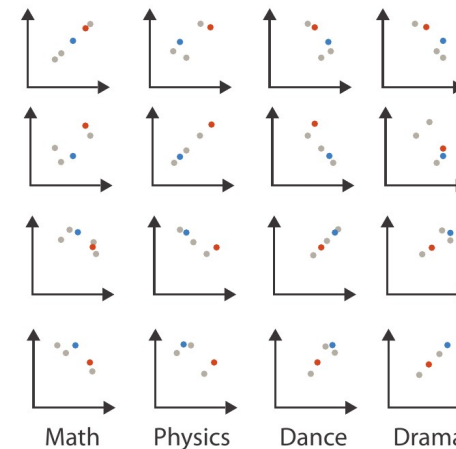
Scales to many columns. But only adjacent columns can be compared.

Table

Math	Physics	Dance	Drama
85	95	70	65
90	80	60	50
65	50	90	90
50	40	95	80
40	60	80	90



Scatterplot Matrix



Parallel Coordinates

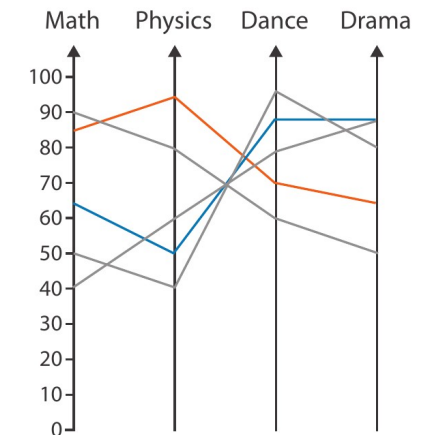
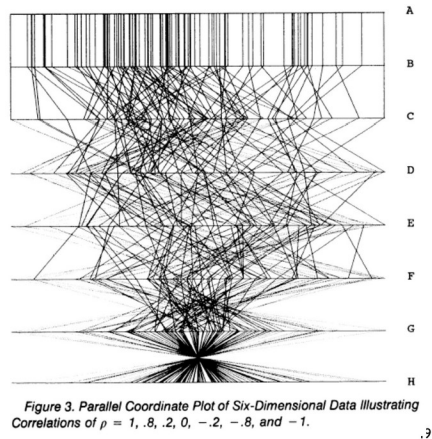


Figure from the SPU Information Visualization course deck, M. Senturk (github.com/spu-dataviz-211), after Munzner VAD.

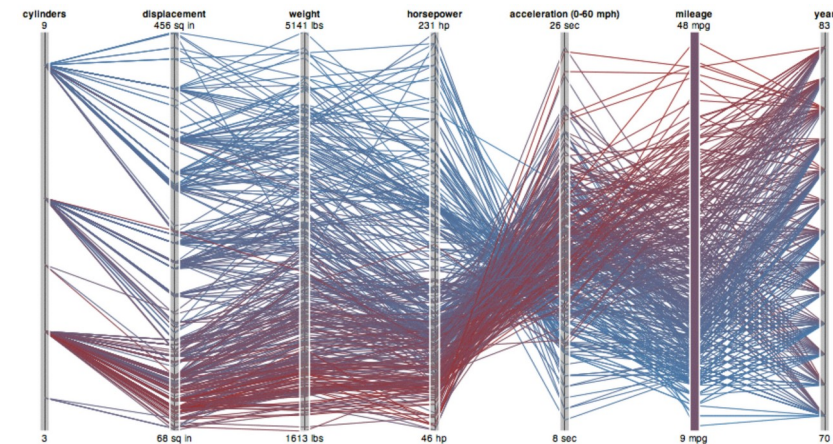
Reading parallel coordinates

What the patterns between two axes actually mean.



Correlation shows up as a pattern

Between any two neighbouring axes, lines that stay roughly parallel mean a positive relationship. Lines that cross in a tight X mean a negative one. A tangled mess means little relationship at all. That is the whole reading skill.

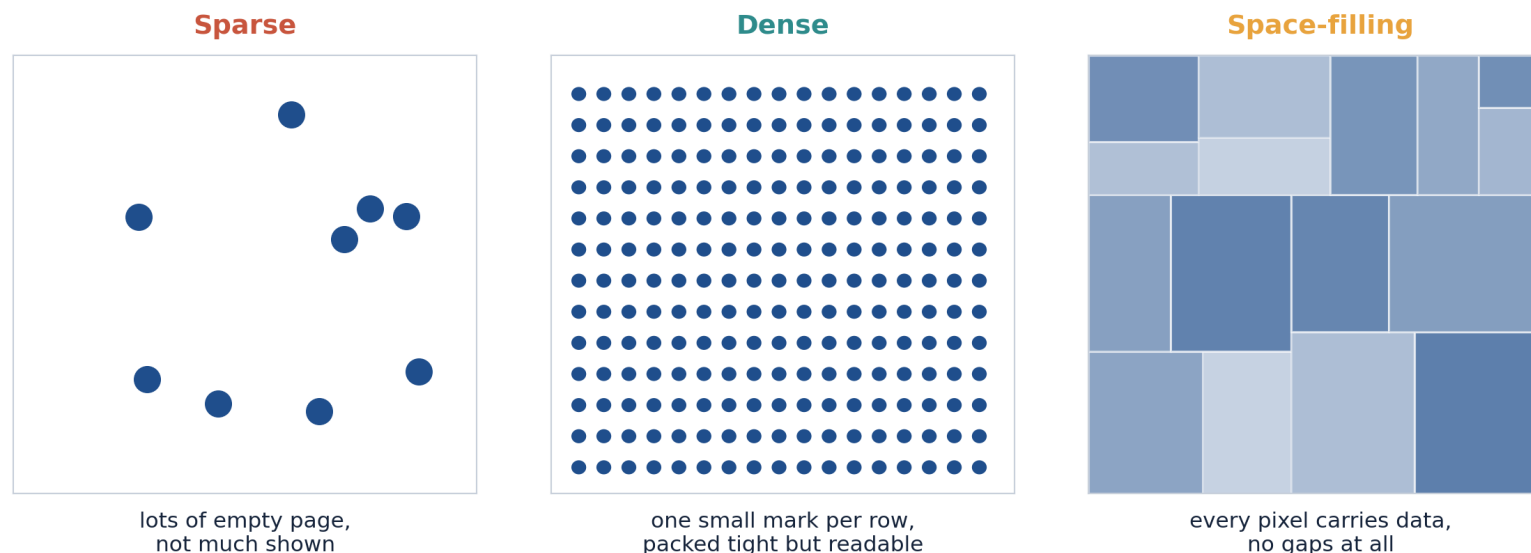


A real one, coloured by a category

Here a car dataset is coloured by number of cylinders. The two coloured bands separate cleanly across several axes at once, which tells you those attributes together distinguish the groups — something no single scatterplot would show as directly.

Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

How tightly should you pack the page?



The last arrangement decision. A sparse layout wastes the reader's screen and forces scrolling. A space-filling one uses every pixel but leaves nothing to rest the eye on.

Dense in the middle is usually what you want: one small mark per row, packed tightly, with enough separation that marks remain distinguishable. This is the information-density idea from Week 1 made concrete — and it is a judgement call that depends on the task, not a formula.

After Munzner VAD Ch. 7.

Dense displays done well

A view of a large software codebase. Every line of code is one pixel row, every file a column.

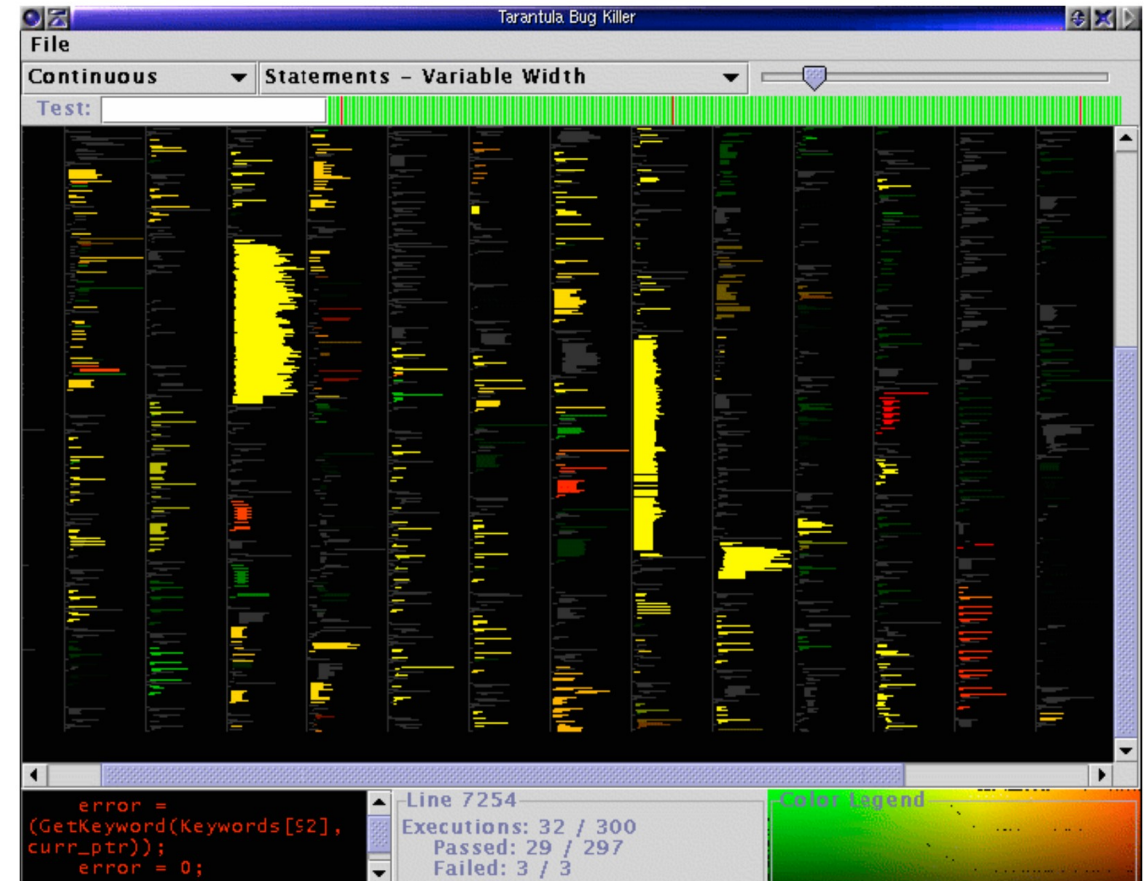
There is no way to read any individual line. That is not the point.

The point is that the overall texture — where the yellow concentrates — tells you which parts of the system the failing tests touch. You then zoom into those.

This is overview-first thinking, and it is the natural bridge into tonight's second half.

IN ONE LINE

At extreme density, individual values vanish and texture becomes the signal.



Tarantula, Jones, Harrold & Stasko, ICSE 2002. Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

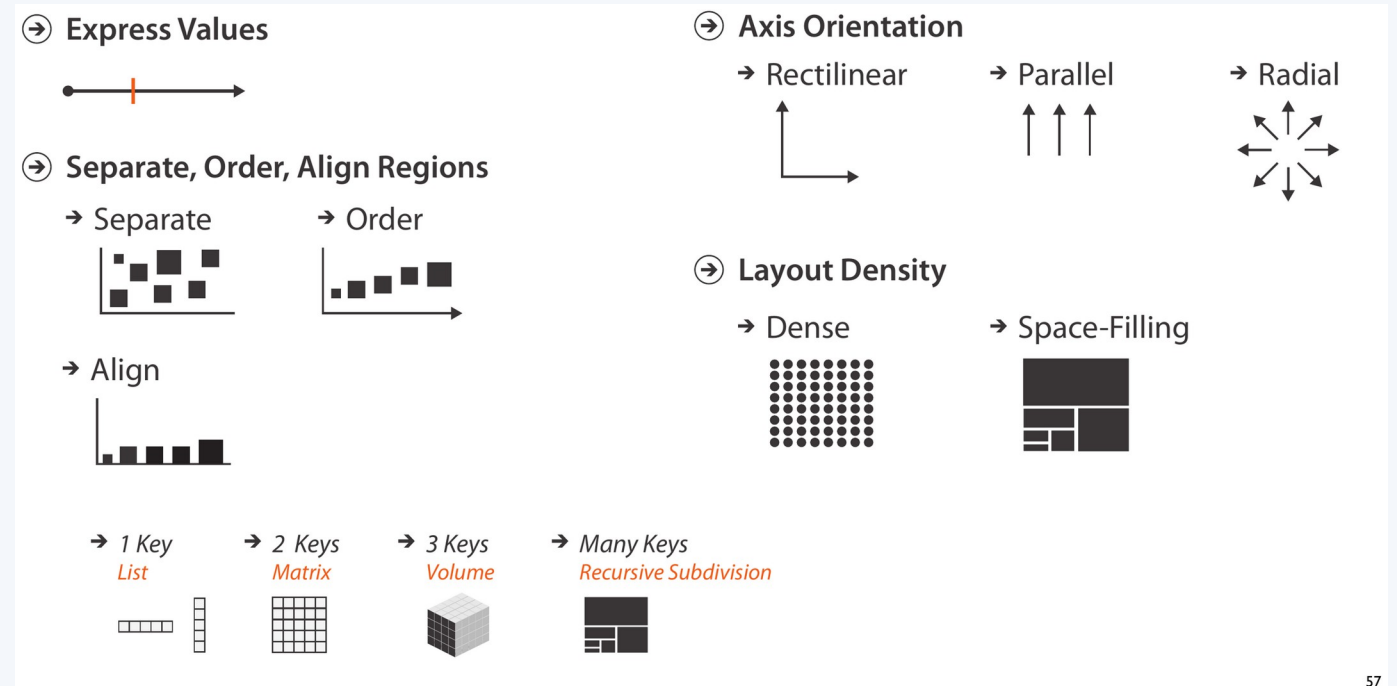
Everything from Week 3 on one page

Express values for no keys. Separate, order and align for regions. Then the axis orientation choice and the density choice.

Down the bottom: how many keys you have, and what each count gives you.

Keep this open while you work. For any chart you build or critique, you should be able to point at each row and say which option you chose.

Posted full size on Blackboard.



57

Munzner, *Visualization Analysis and Design*, Ch. 7 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.



04

WEEK FOUR

What happens when you can touch it?

Everything so far assumed one fixed picture. Now the reader gets a mouse. That single change creates a whole second design problem — and solves a lot of the ones we just hit.

Three ways one view can change



Three ways a single view can change under the user's hand.

If you keep a single chart on screen and let the reader act on it, there are only three categories of thing they can do.

They can change what is drawn — a different encoding, a different sort order, a different filter. They can select something, which is how almost every other interaction begins. Or they can move the camera, changing which part of the data is in frame without changing how it is drawn.

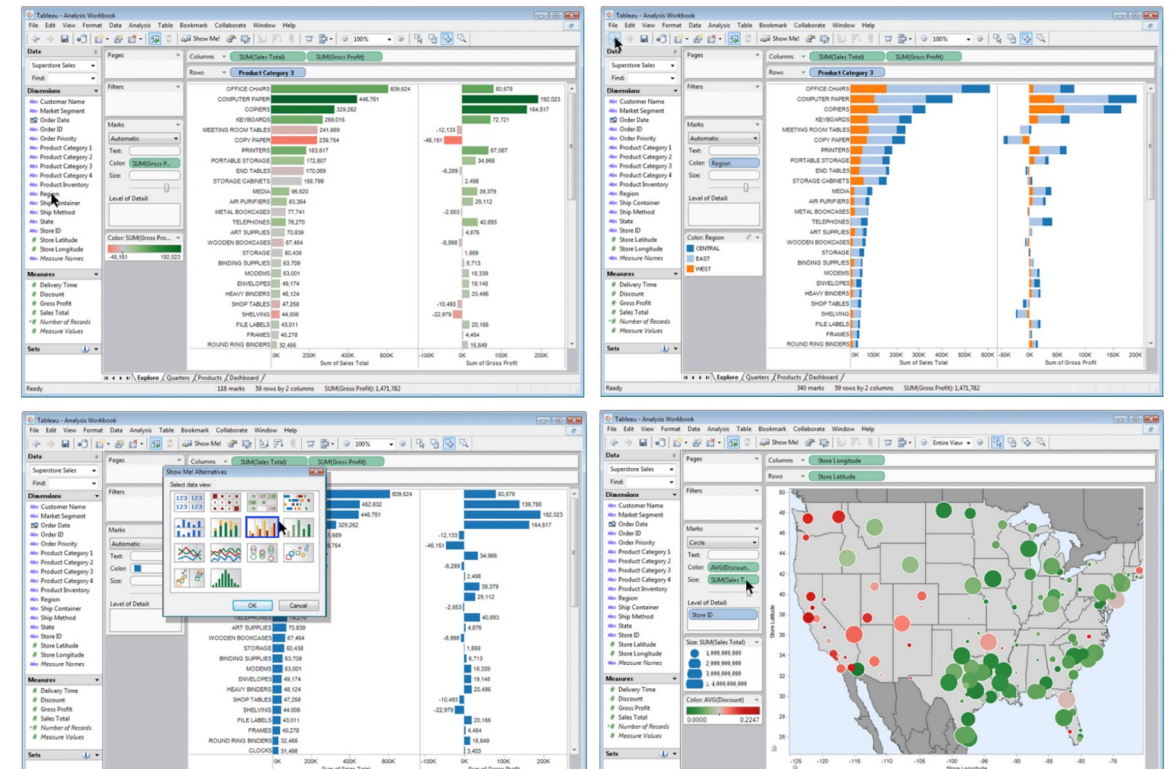
Change: swap the encoding itself

The most dramatic kind of change: keep the data, replace the chart.

These four screens show the same table as a bar chart, a stacked bar, a different chart type chosen from a menu, and a map.

This matters because of the Week 2 lesson — different encodings support different tasks. Rather than guessing which task the user has, let them switch.

Tableau is built around this idea, which is one reason it dominates the tool market.



IN ONE LINE

If one chart cannot serve two tasks, let the reader swap between them.

Made using Tableau, tableausoftware.com. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Change: widgets and controls

The familiar approach: sliders, dropdowns, checkboxes, buttons. Here a year slider drives a map of US railroad growth.

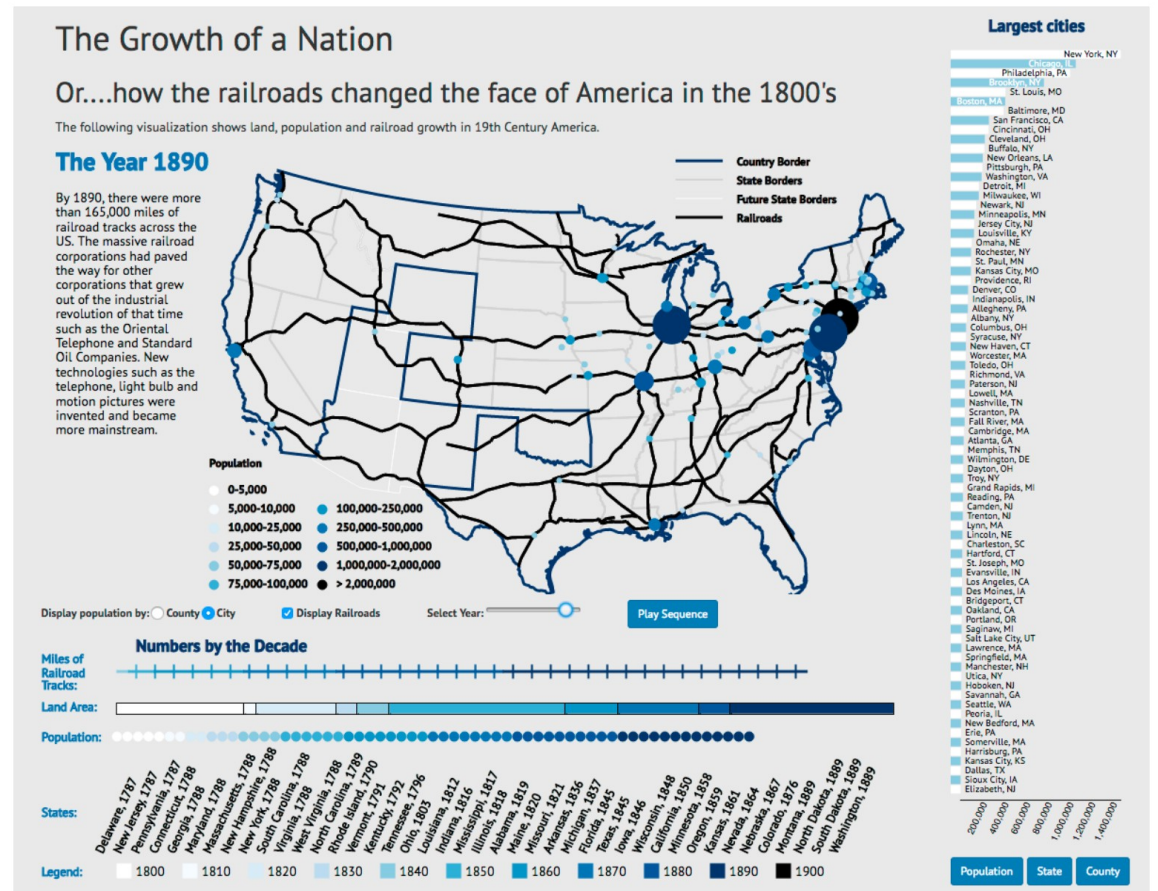
The advantage is discoverability. A labelled slider announces what it does; a hidden gesture does not.

The cost is screen space. Every control is pixels not spent on data, which is the Week 3 density trade-off again.

The design question is whether controls sit in a separate panel or are interleaved with the chart itself. Separate is tidier; interleaved keeps the control next to what it affects.

IN ONE LINE

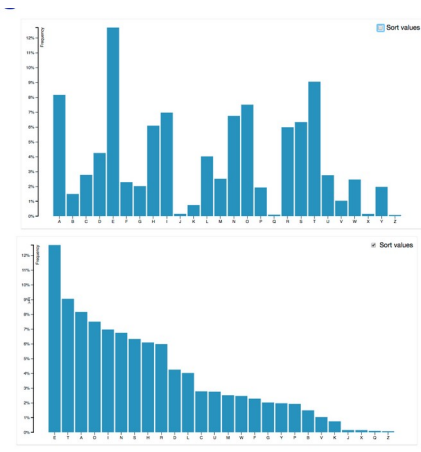
Controls are honest but expensive. Spend the space deliberately.



The Growth of a Nation, laurenwood.github.io. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

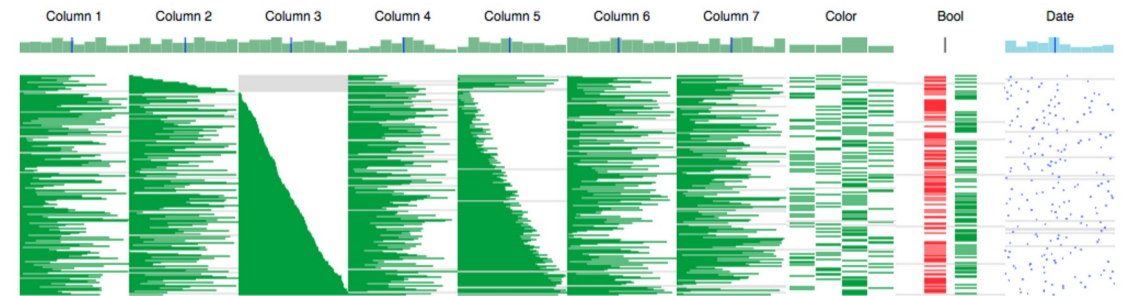
Change: let the reader re-sort

Week 3 said ordering matters enormously. Interaction says: do not pick one order, pick all of them.



One click to sort

The simplest and most valuable interaction in any bar chart. Unsorted supports lookup by name; sorted supports ranking. One checkbox gives the reader both, and costs almost nothing to build.



Sort a wide table by any column

The same idea on a wide table. Click a column and every row reorders by it. Because the rows stay aligned across all columns, a column that sorts into a clean gradient alongside the one you clicked is correlated with it — you can find relationships by sorting.

Left: Bostock, Sortable Bar Chart. Right: carlmanaster.github.io/datastripes. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Change: move the baseline

Week 3's stacked bar problem: only the bottom segment sits on a common baseline, so the others are hard to compare.

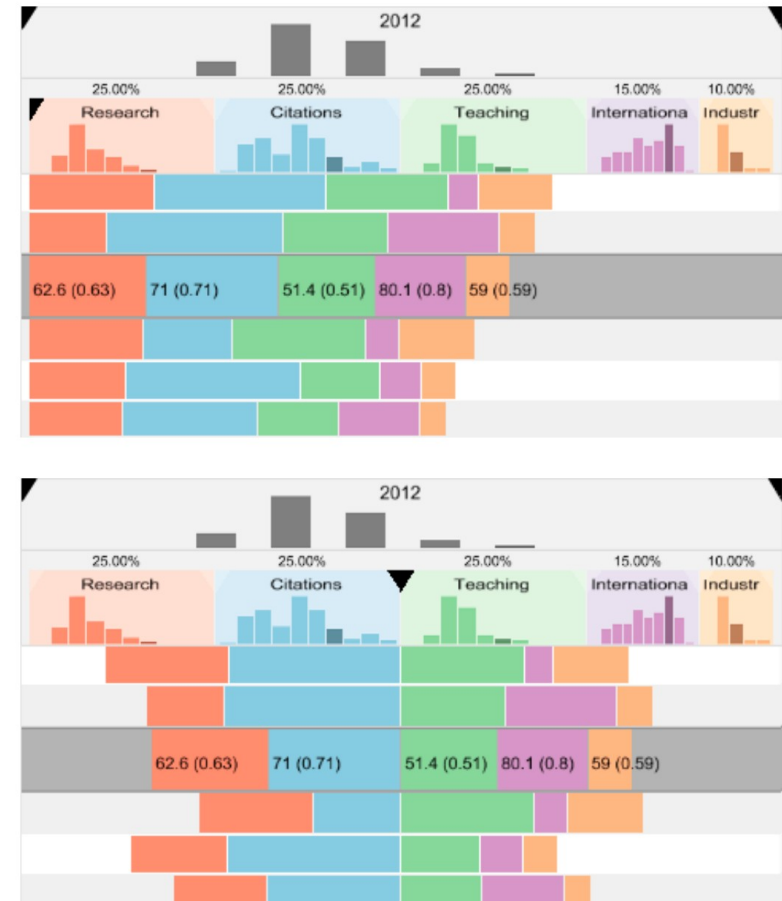
LineUp solves it by letting the reader pick which segment to align on. Click a different segment and every bar re-anchors to it.

Now the comparison that was hard becomes the easy one — and you can switch again for the next question.

This is a neat demonstration of the general pattern: take a fixed design decision from Week 3 and hand it to the user.

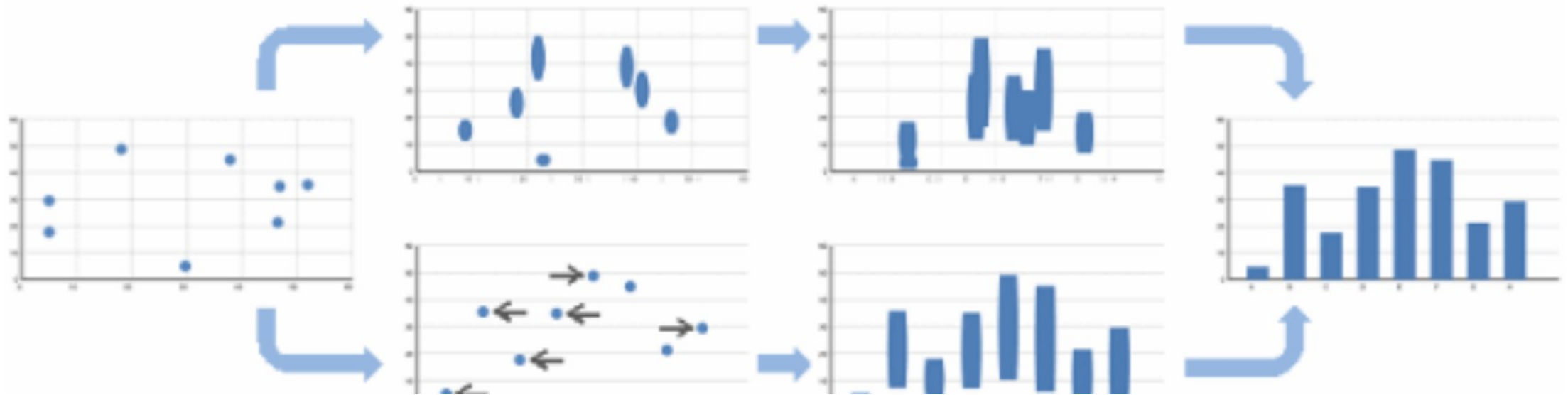
IN ONE LINE

Any Week 3 decision you could not settle is a candidate for an interactive control.



LineUp: Gratzl, Lex, Gehlenborg, Pfister & Streit, IEEE TVCG 19(12), 2013. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Changing smoothly: animated transitions

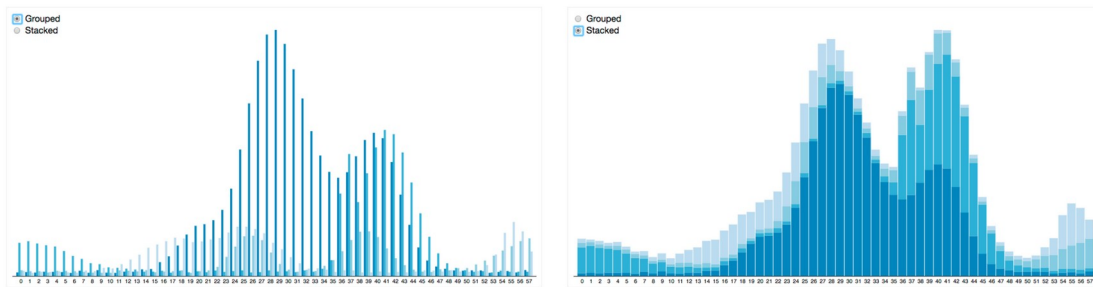


When the view changes, you can cut instantly or you can interpolate smoothly between the two states. Smooth is usually better, for one specific reason: it lets the viewer track individual items across the change.

That matters because of the Week 1 change-blindness finding. With a jump cut, the viewer has to remember where everything was. With a smooth transition, the eye follows the motion and no memory is required. Staging the change in steps rather than moving everything at once helps further.

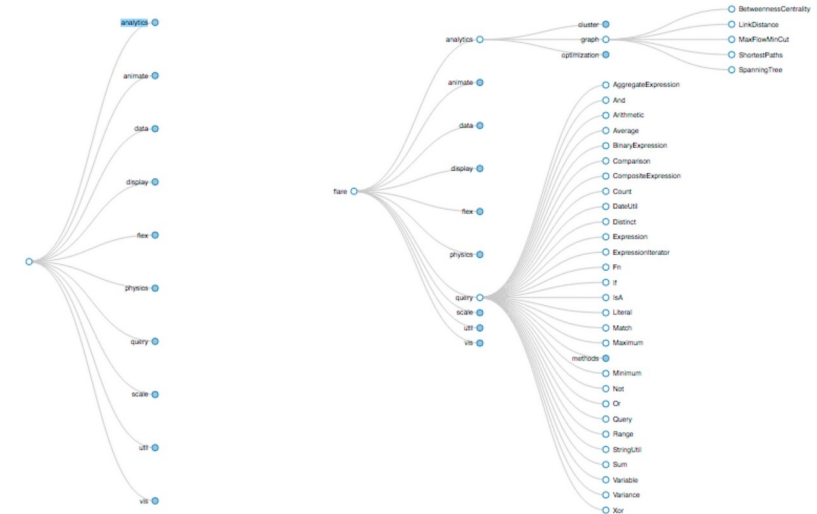
Heer & Robertson, Animated Transitions in Statistical Data Graphics, IEEE TVCG 13(6), 2007. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Two things transitions are good at



Switching encoding

Stacked to side-by-side, animated. This is exactly the Week 3 dilemma, resolved: the reader gets both charts and the transition makes it obvious that the same bars are being rearranged rather than replaced with different data.



Opening up detail

Expanding and collapsing a tree. Only part of the structure is drawn at any time, and the transition shows where the new branches came from. Without it, the layout would appear to change at random.

Bostock, *Stacked-to-Grouped Bars and Collapsible Tree*. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

A quiz: what changed between these?

Four views of the same US federal budget proposal from one New York Times graphic.

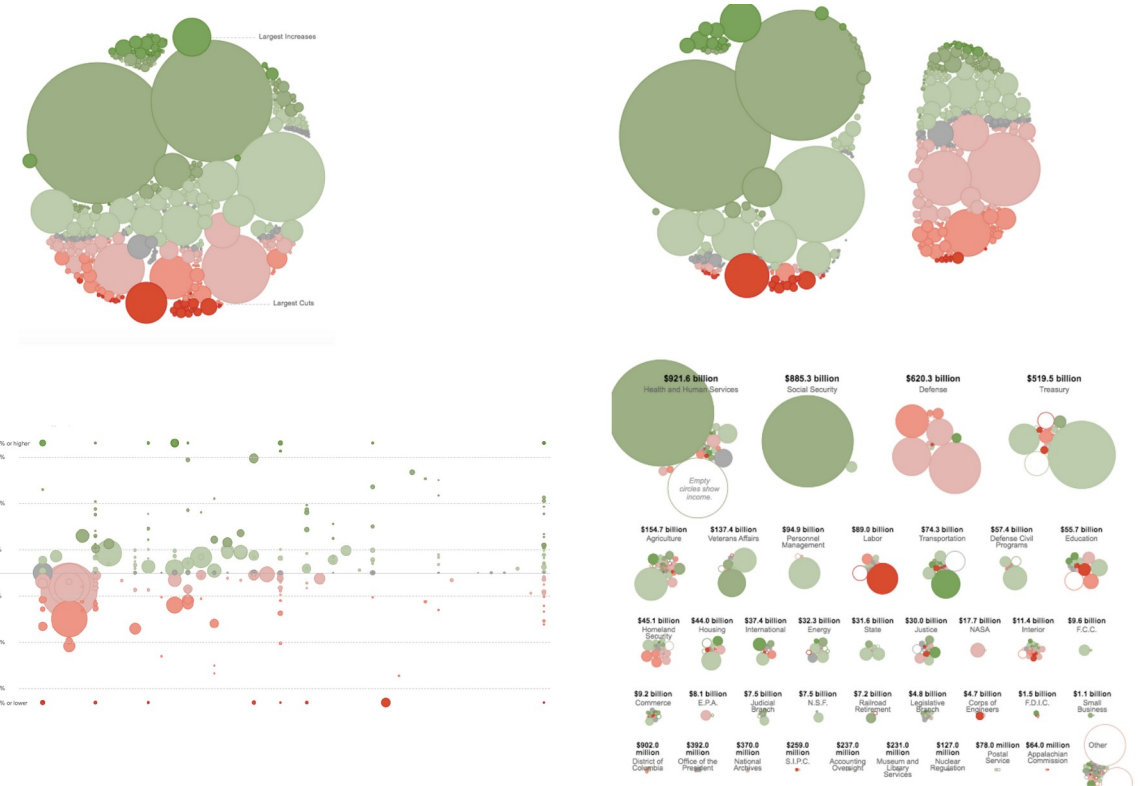
Take thirty seconds. What is different between them — the data, the encoding, the arrangement, or the grouping?

The answer: the data never changes. What changes is the arrangement and the grouping — the same circles are regrouped by department, laid out by percentage change, and so on.

Each arrangement makes a different question easy, which is the entire argument of tonight's first half.

IN ONE LINE

Four arrangements, one dataset, four different questions answered.



NYT, 2013 budget proposal graphic, archive.nytimes.com. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Select: the decisions hiding behind a click

Selection looks trivial and is not. Every one of these is a design choice someone has to make.

How?

Click, tap, or hover? Hover is cheap and fast but does not exist on touchscreens. Click is deliberate but slower. Many designs need both.

How many?

Does a new click replace the selection or add to it? Can the user select nothing? Shift-click and option-click give you more selection types, at the cost of discoverability.

What kind?

Some designs need more than one kind of selection at once — a source and a target in a network, for instance. Each kind needs its own visual treatment.

Get selection wrong and every interaction built on top of it feels wrong too.

Highlighting: showing what is selected

Selection is what the user does; highlighting is how you show it. They are related but separate decisions.

The obvious move is to change the item's colour — but if colour already encodes something, you have just destroyed that encoding.

Safer options: add an outline, thicken an existing outline, or change the mark's shape. All of these add rather than replace.

Motion is the unusual one. Avoid it in a single view, where it is distracting. In a multi-view design it can be justified, because it draws the eye to the other view.



Colour already carries the group in this chart. Highlighting with colour throws that away — the other three add a channel instead of overwriting one.

IN ONE LINE

Highlight by adding something, not by overwriting a channel that already means something.

Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Tooltips: useful, and more limited than you think



What they are good for

Detail on demand. The exact number behind a bar, the full name behind an abbreviation, a definition behind a label.

They cost no screen space until they are needed, which makes them attractive on dense layouts.



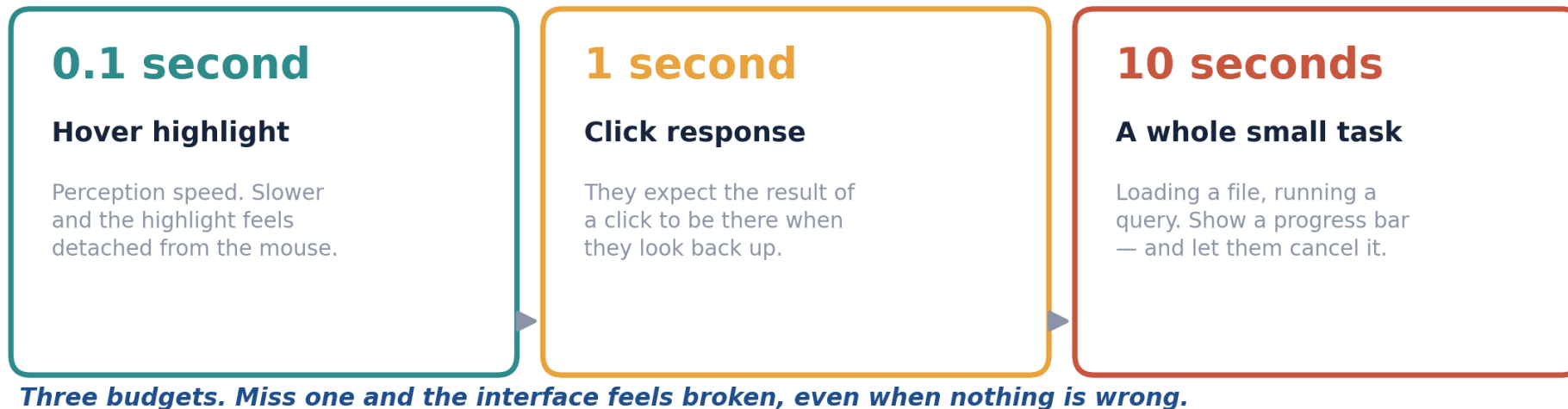
What they cannot do

Give an overview. A tooltip shows one item at a time, so any task that requires comparing or scanning is impossible through tooltips alone.

And people miss them. Gregor Aisch of the NYT puts it bluntly: assume nobody will see your tooltip. If it matters, encode it visibly.

A tooltip is a supplement to a good encoding, never a substitute for one.

However you do it, it has to be fast



Three timing budgets, and they come from human perception rather than from engineering taste.

Under about a tenth of a second, feedback feels attached to the action — that is the budget for hover highlighting. Around a second is what people will tolerate after a deliberate click before they start to wonder. Ten seconds is a whole small task, and anything that long needs a progress bar and a cancel button.

After Munzner VAD Ch. 11.

Navigate: moving the camera

Changing which items are visible, without changing how they are drawn.

➔ Navigate

➔ Item Reduction

➔ Zoom
Geometric



➔ Pan/Translate



Zoom and pan

The camera metaphor. Pan moves sideways, zoom moves closer. Geometric zoom just magnifies — everything gets bigger. Semantic zoom is cleverer: as you zoom in, the representation itself changes, so a dot might become a labelled box.

➔ Navigate

➔ Item Reduction

➔ Zoom
Geometric or Semantic



➔ Pan/Translate



➔ Constrained



Let the design do the driving

Unconstrained navigation is easy to build and hard to use — people overshoot, get lost, and cannot find their way back. Constrained navigation computes the destination for you: click a thing and the view animates to frame it properly. Far easier to use, more work to build.

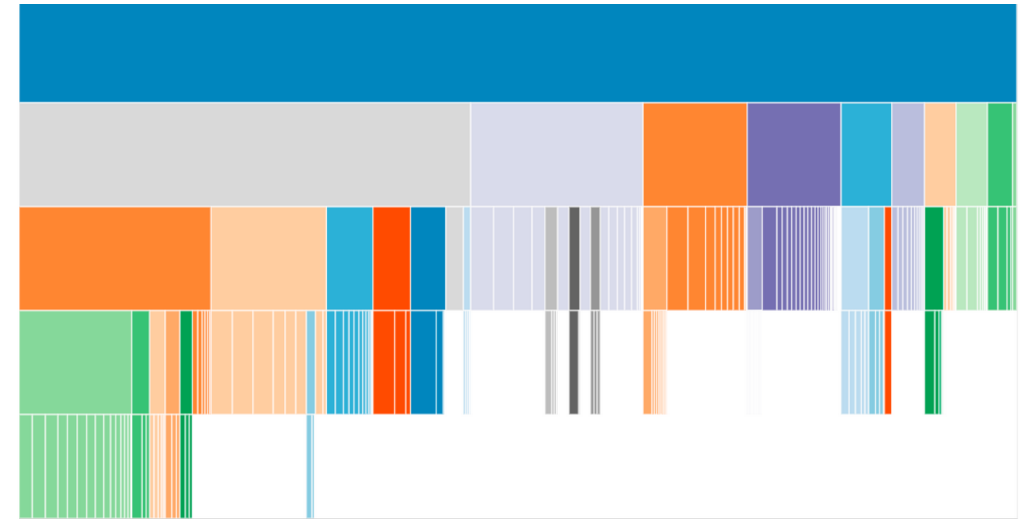
Two kinds of zoom, side by side

Zoom to Bounding Box



Shapes preserved

Click a state and the map flies to frame it. This is the simple case: only the viewport changes, so California still looks like California. The user's mental model survives the transition intact.



Shapes change

An icicle plot of a hierarchy. Click a block and it expands to fill the width, so its children become readable. Here the shapes genuinely change — proportions are recomputed — and that is why the animated transition matters so much.

Bostock, Zoom to Bounding Box and Zoomable Icicle. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Scrollytelling: navigation by scrolling

Scroll down the page and the graphic advances through a prepared sequence. Hugely popular in news graphics.

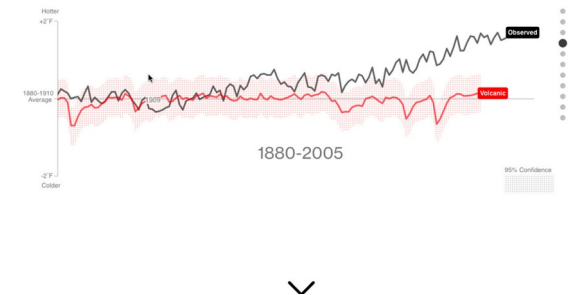
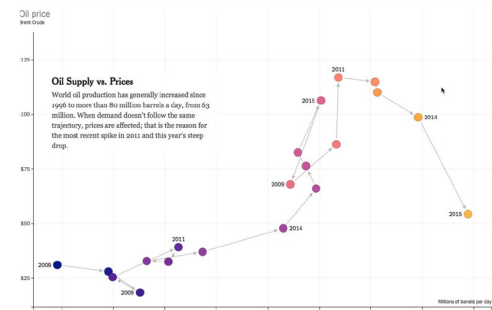
The appeal is real. Scrolling is the one interaction every reader already knows, and it is linear, so there is no interface to learn and no way to get lost.

The criticisms are also real. Full-screen modes hide the affordance, scroll-jacking takes control away from the reader, and a continuous gesture driving discrete steps often feels imprecise.

It works best when there genuinely is one story to tell in one order.

IN ONE LINE

The best interaction is often the one the reader already knows how to do.



NYT oil prices; Bloomberg What's Warming the World. See eagereyes.org/blog/2016/the-scrollytelling-scurge. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

What interaction buys, and what it costs



The case for

It is the main advantage a screen has over paper.

It supports exploration — you can change course as you learn. It lets one design serve several tasks by switching encodings. And animated transitions genuinely help people stay oriented.

It resolves most of the dilemmas from the first half of tonight.



The case against

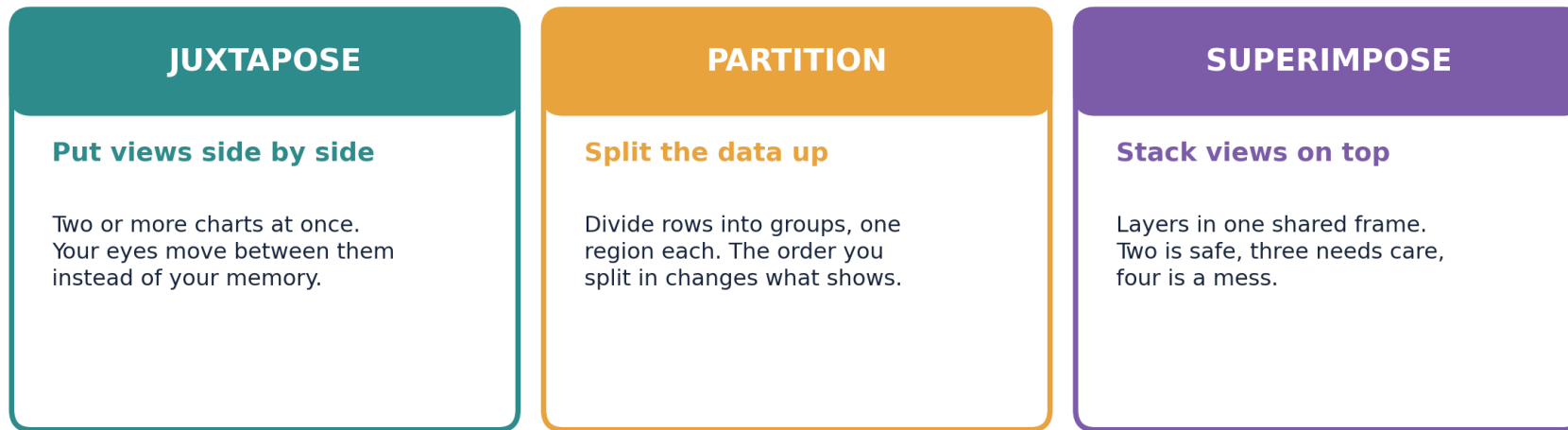
Interaction takes time. In the worst case you have replaced a visual search with a human-powered one, item by item.

Remembering previous states costs the reader effort. Controls cost screen space, and hidden controls cost discoverability.

And the hard number: NYT logs suggest about 90% of readers never interact beyond scrolling.

Design for the reader who never clicks anything. Then reward the one who does.

Three ways to use more than one view



Three ways to use more than one view — or more than one layer.

The other half of tonight's complexity strategy. Instead of changing one view over time, use several at once.

You can put views side by side, split the data between regions, or stack layers in a single frame. The underlying argument for all three is the same, and it is a Week 1 argument: moving your eyes between two things on screen is cheaper than remembering what the first one looked like.

Side by side: what do the views share?

Once you have two views, three separate things can be shared or not shared between them.

ENCODING — are they the same type of chart, or different types?

DATA — do they show all the same rows, some overlap, or completely different rows?

NAVIGATION — does panning or zooming one move the other too?

Those three answers define every multiple-view design there is, which is why the next few slides all look like variations on one idea.

IN ONE LINE

Encoding, data, navigation. Answer those three and you have specified the design.

→ Share Encoding: Same/Different

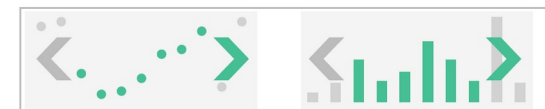
→ *Linked Highlighting*



→ Share Data: All/Subset/None



→ Share Navigation



Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Linked highlighting: the one to remember

Several different charts of the same rows. Select in any one of them, and the same rows light up in all the others.

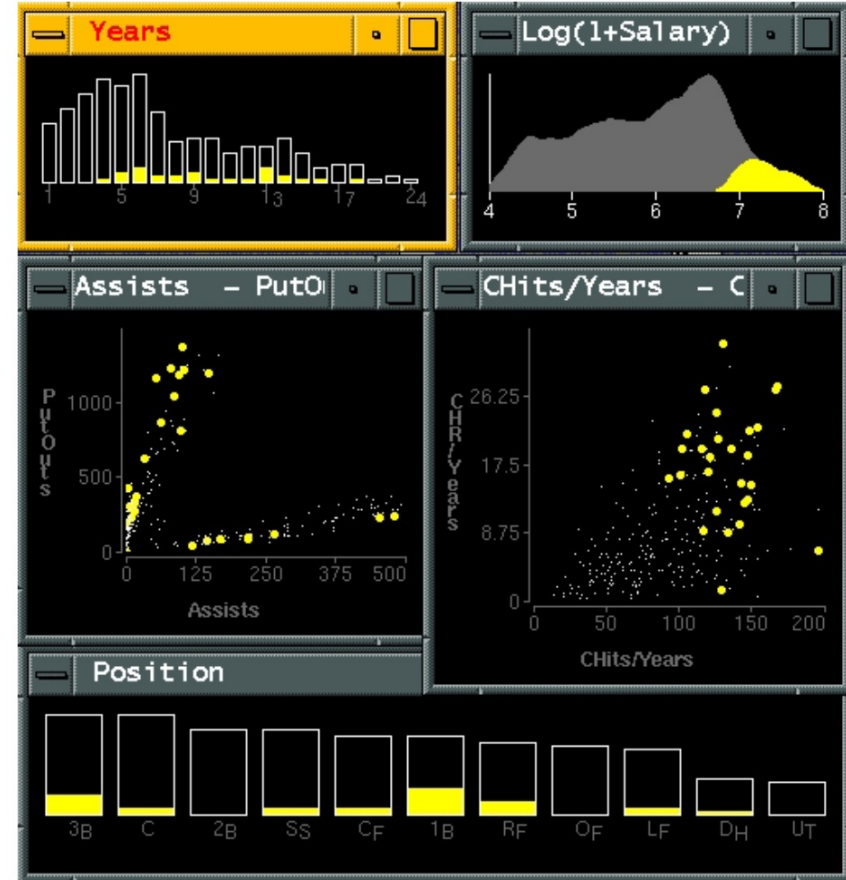
The yellow here is one selection, shown simultaneously as a slice of a histogram, a cluster in a scatterplot, and a group in a bar chart.

This answers a question no single chart can: where do the items that are extreme in one attribute sit in all the others?

It is also called brushing and linking, and it is probably the single most powerful interaction idiom in the field.

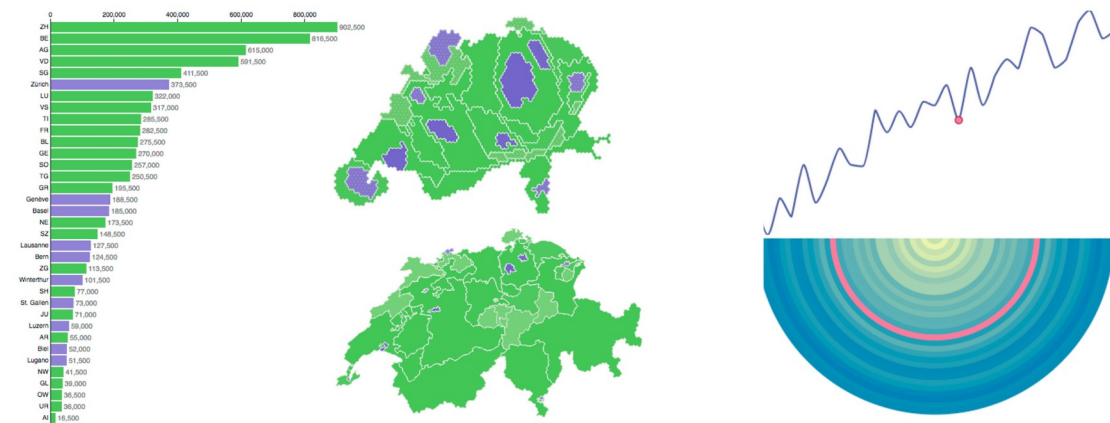
IN ONE LINE

Select once, see it everywhere. Nothing else reveals multi-attribute structure so directly.



EDV: Wills, *Visual Exploration of Large Structured Datasets*, NTTs 1995. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Linked views in practice



Map and bar chart, linked

A Swiss population cartogram beside a sorted bar chart. Hover a canton in either view and it highlights in both. The bar chart gives you ranking, the map gives you geography, and the link lets you move between the two questions freely.



A full linked dashboard

A basketball analytics tool. Shot chart, frequency curves, and comparison panels all showing the same player. Change the player and everything updates together; hover a zone and it highlights across the panels.

Left: ralphstraumann.ch. Right: buckets.peterbeshai.com. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Overview plus detail: the most common pattern

Same encoding, same data, different zoom level. A small view showing everything, and a large view showing part of it.

The small one tells you where you are and what else exists. The large one lets you work.

Note the box in the small map marking the detail view's extent — that is what makes the pairing legible rather than confusing.

This solves the oldest problem in navigation: zoom in far enough to be useful and you lose all sense of context.

IN ONE LINE

Context in the small view, work in the big one, and a box linking them.



Cockburn, Karlson & Bederson, A Review of Overview+Detail, ACM Computing Surveys 41:1, 2008. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Overview plus detail, for time series

The same pattern applied to a long time series, and you have met this one — any stock chart works like it.

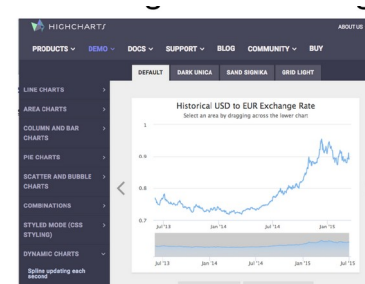
A full-range strip at the bottom, a detail chart above. Drag a window across the strip and the detail view follows.

The sharing here is unidirectional: the overview drives the detail, not the other way round.

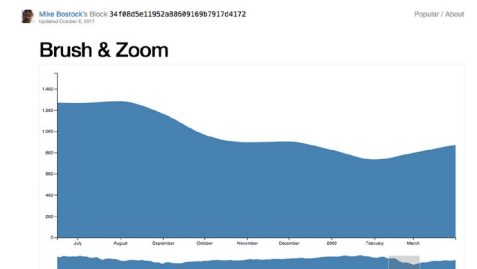
It is a particularly good fit for time, because a long series is exactly the case where you need both the shape of the whole decade and the detail of one week.

IN ONE LINE

For long time series this is close to the default answer.



<https://www.highcharts.com/demo/dynamic-master-detail>



Highcharts master-detail demo; Bostock, Brush & Zoom. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Small multiples: the same chart, many times

Same encoding, no shared rows. One small copy of the chart per group, laid out in a grid.

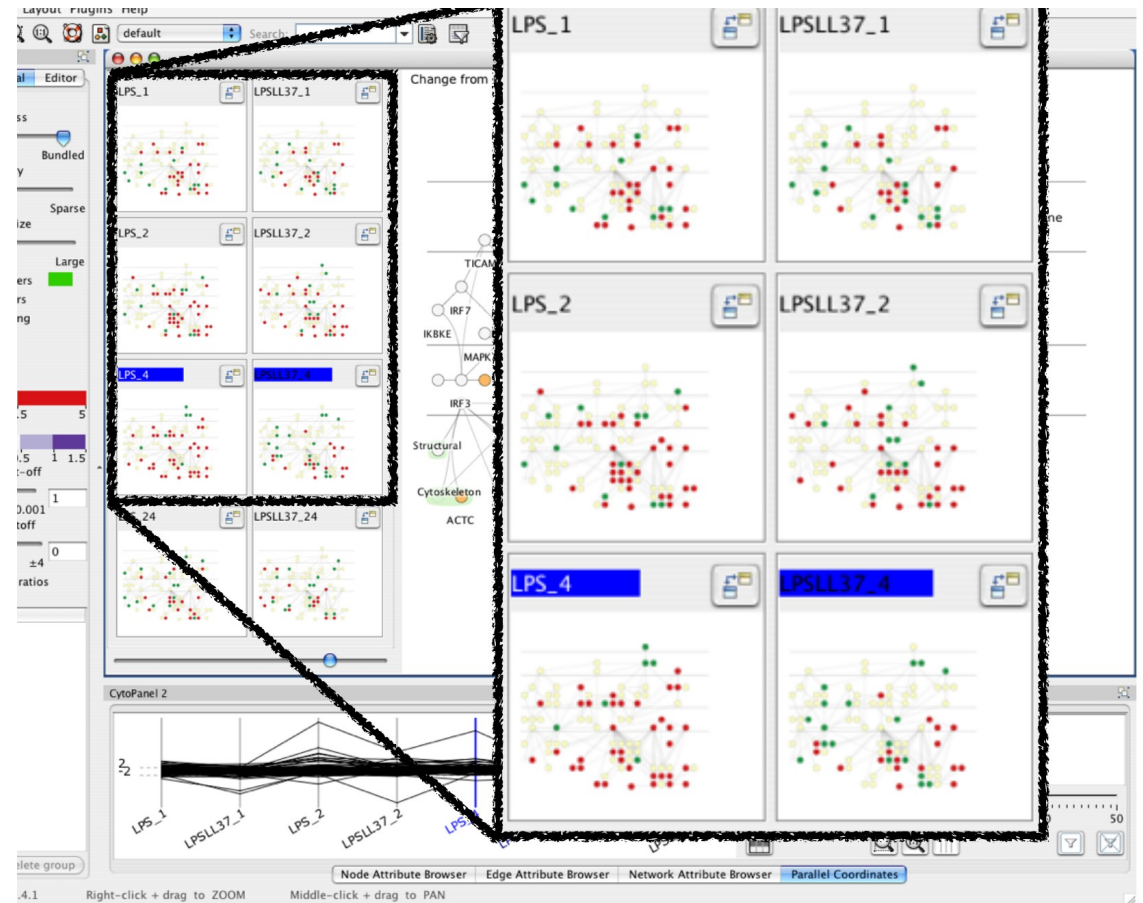
Because every panel is identical in form, your eye compares them almost automatically — differences in shape jump out.

This is the honest alternative to animating through the groups, and it is almost always better, for the Week 1 reason: comparison across time demands memory, comparison across space does not.

The constraint is size. Each panel is small, so only coarse patterns survive.

IN ONE LINE

If you are tempted to animate through categories, try small multiples first.



Cerebral: Barsky, Munzner, Gardy & Kincaid, IEEE TVCG 14:6, 2008. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Which multiple-view design have you built?

	ALL the data	SOME of the data	NONE shared
SAME chart type	Redundant same chart, same data — why would you?	Overview + detail a small map and a big one; click the small to move the big	Small multiples a grid of the same chart, one per group
DIFFERENT chart types	Multiform two different charts of the same rows, linked together	Multiform + overview-detail the most powerful, and the most work to build	No linkage nothing shared — not really a coordinated view at all

Two questions decide which one you have built: do the views use the same chart type, and do they show the same rows?

Two of the three sharing questions — same encoding or different, and how much data is shared — produce a grid with a name in each cell. Every design in this section lands somewhere on it.

The two greyed corners are the degenerate cases. Same chart and same data is simply redundant. Different charts and no shared data is not a coordinated view at all, just two pictures on a page. Everything useful lives in between.

After Munzner VAD Ch. 12.

So when is animation the wrong answer?

Earlier we praised animated transitions. Here is the limit of that praise.

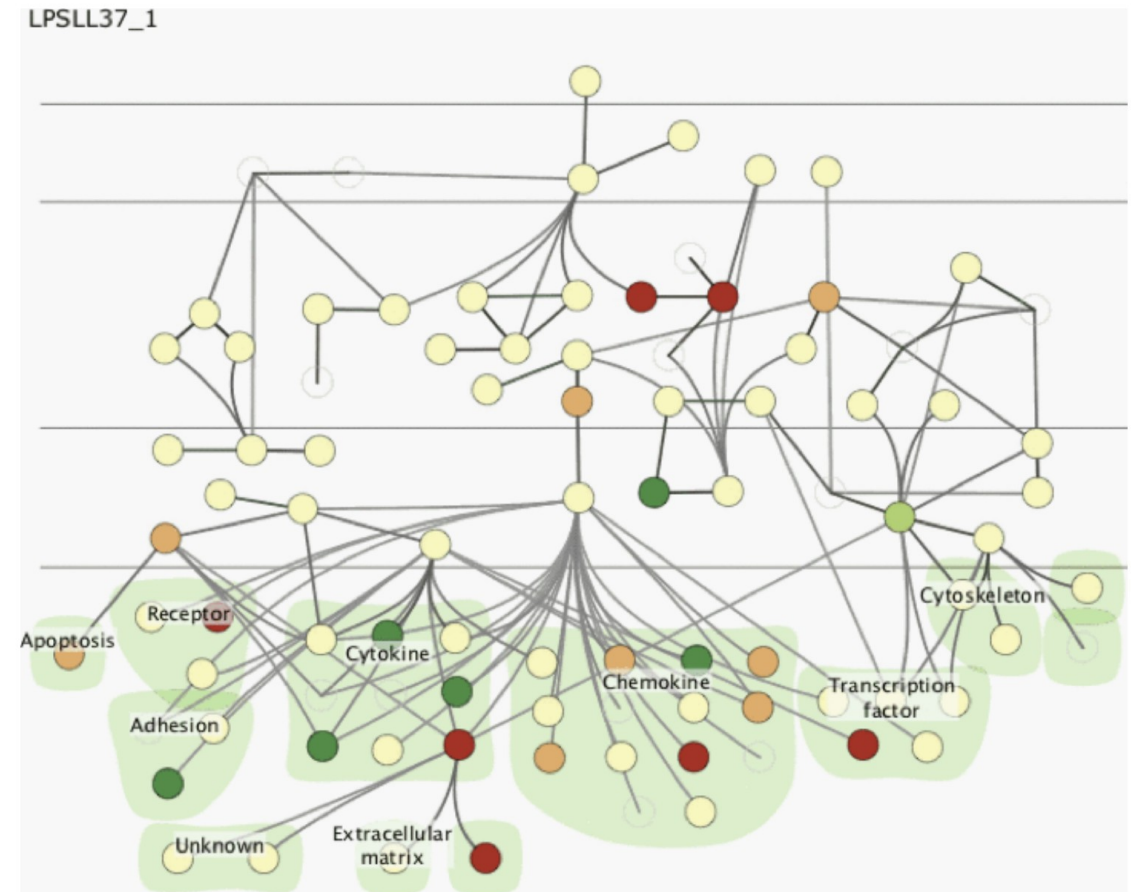
Animation works when the change is small, contiguous and coherent — a group of items moving together, a smooth re-layout.

It fails when the frames are very different, when changes are scattered across the whole display, or when the user must compare something in frame one with something in frame twenty.

In those cases juxtaposition wins. Put the frames side by side and let the eyes do what memory cannot.

IN ONE LINE

Animate smooth, local change. Juxtapose anything that needs comparing.



Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

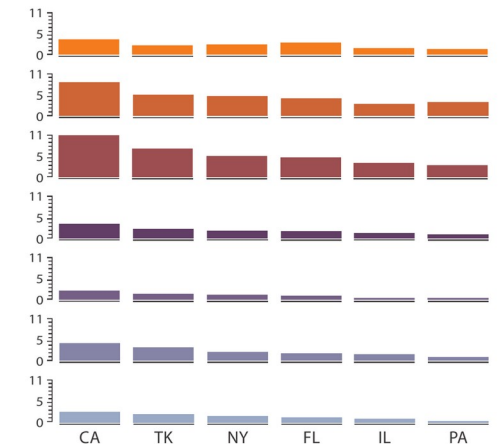
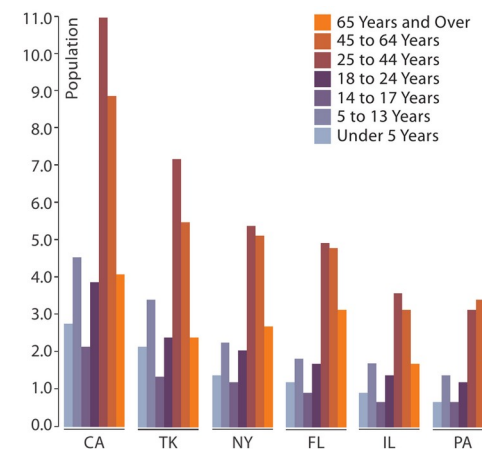
Partition: split the data into regions

The second facet move. Divide the rows into groups by some attribute and give each group its own region.

The two panels show identical data split two different ways. On the left, one chart split by state, with age groups inside each. On the right, separate charts split by age group, with states inside each.

The left makes comparison within a state easy and across ages hard. The right does the opposite.

Which attribute you split on first decides which comparison is easy. There is no neutral choice.



55

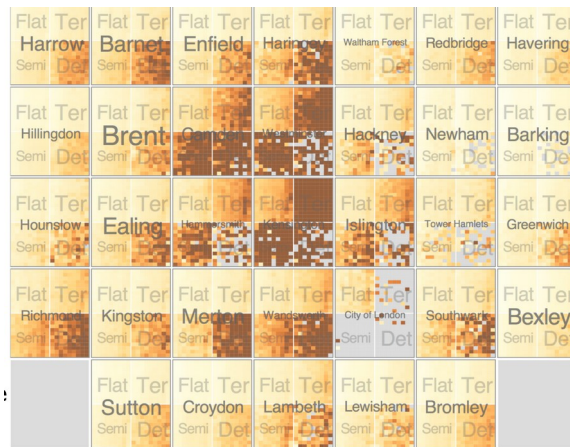
IN ONE LINE

The order you split in is a design decision with real consequences.

Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Splitting again, and again

London house prices, partitioned by neighbourhood, then by property type, then by time.



Neighbourhood first

Split by neighbourhood, then by type, then years as rows and months as columns inside each cell, coloured by price. You can see which neighbourhoods are expensive, and where detached homes command the biggest premium.



Property type first

Identical data, splits reversed. Now the top-level grouping is property type, and the question that becomes easy is a different one: within a given type, which neighbourhoods behave inconsistently?

HIVE: Slingsby, Dykes & Wood, IEEE TVCG 15:6, 2009. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

And if the regions are sized by the data

Same recursive splitting, with one change: each region's size is proportional to how much data it contains, rather than every region being equal.

Do that and you have derived a treemap — which is where the treemaps from Week 1 actually come from.

The advantage is that large groups get the space they deserve and small ones do not waste the page.

The cost is that the regions are no longer a tidy grid, so comparing positions across the layout gets harder.

IN ONE LINE

A treemap is just recursive partition with regions sized by the data.



HIVE: Slingsby, Dykes & Wood, IEEE TVCG 15:6, 2009. Munzner, Visualization Analysis and Design, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Superimpose: stack views in one frame

The third facet move. Instead of putting views beside each other, put them on top of each other in one shared frame.

Each layer is a set of marks spread over the whole view, and each must be visually distinguishable from the others.

The hard constraint is how many layers you can manage. Two is comfortable. Three needs careful design. Four is usually a mess.

The way to distinguish them is to use different, non-overlapping channels for each layer — which is a direct application of Week 2's separability idea.

IN ONE LINE

Two layers is safe. Past three, use side-by-side instead.

➔ Superimpose Layers



Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Layering done well: foreground and background

A road map. Roads are the foreground layer — strong hue, varied width, high contrast against what is behind them.

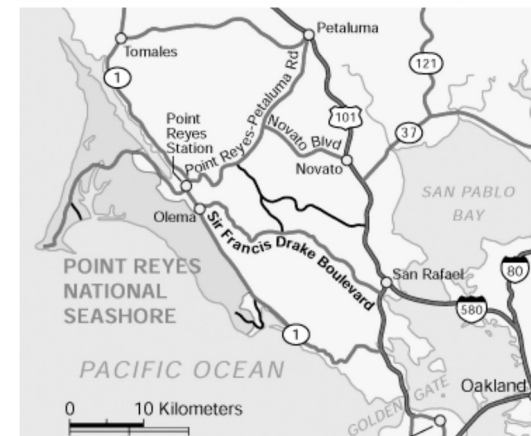
Regions are the background layer — desaturated colours for water, parks and land. Present, but quiet.

The reader can choose to attend to either, and the design does not fight them.

The test is in the second panel: convert it to greyscale. If the layers still separate, the luminance contrast is doing the work and the design is robust. Maureen Stone calls this getting it right in black and white.

IN ONE LINE

If it works in greyscale, it works. Check before you ship.



Stone, *Get it right in black and white*, 2010. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

How many lines can one frame hold?

A different question from how many layers: how many individual lines can share one frame?

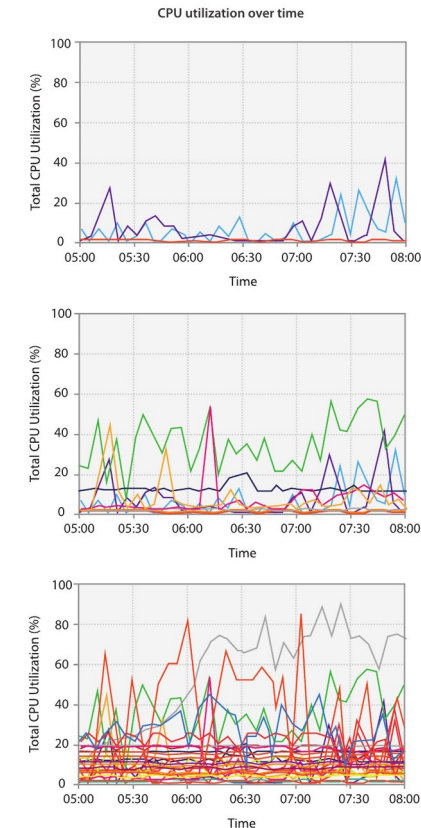
The answer from the three panels is a few dozen, not hundreds. Past that it becomes a texture rather than a set of readable series.

There is also measured guidance on when to superimpose rather than split. Superimposed frames are better for local tasks — finding a maximum, reading one series at one point. Split small multiples are better for global tasks — comparing overall slopes, discriminating between series.

And note the screen-space asymmetry: all the small multiples together occupy the same area as one superimposed frame.

IN ONE LINE

Local questions: superimpose. Global questions: split.



62

Javed, McDonnel & Elmqvist, *Graphical Perception of Multiple Time Series*, IEEE TVCG 16:6, 2010. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Trellis plots: partition and superimpose together

The classic barley yield display, and it uses two facet moves at once.

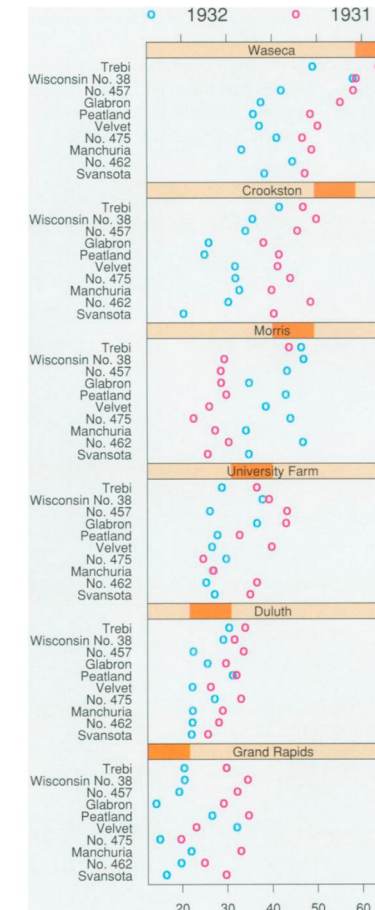
Partition: one panel per site. Superimpose: two years coloured differently within each panel.

The clever part is the ordering. The rows inside each panel are sorted by that variety's median, and the panels themselves are sorted by site median.

That is Week 3 thinking — derive a value, then use it to order — applied to a multi-view layout. The ordering is what makes the anomaly at Morris visible.

IN ONE LINE

Derive a summary value, then use it to order both the panels and the rows inside them.



Cleveland's barley data, after Becker, Cleveland & Shyu. Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

When is it a view, and when is it a mark?

A question that comes up as soon as you start nesting charts inside charts.

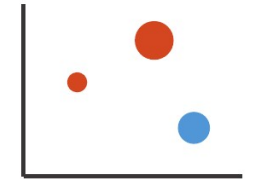
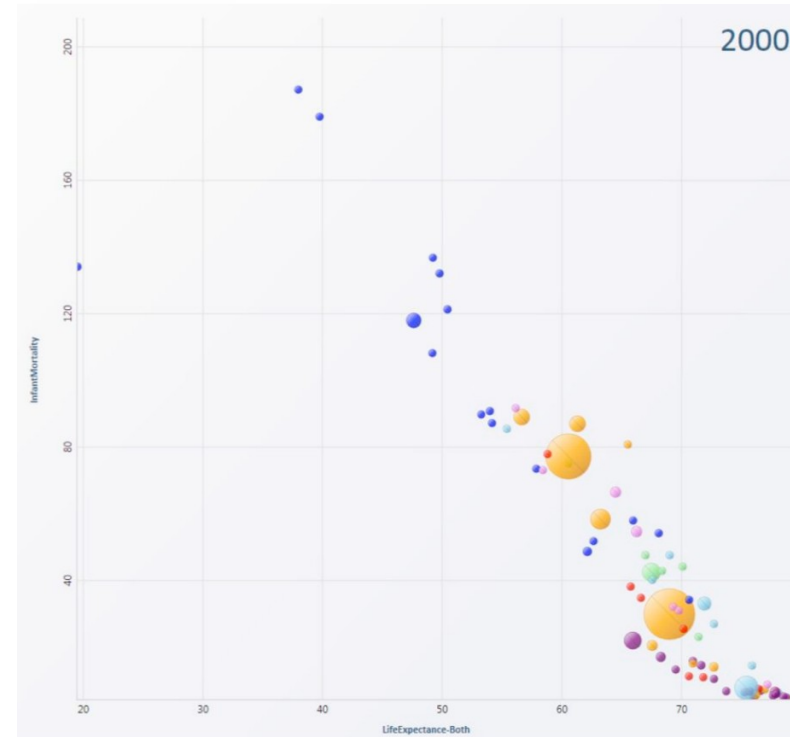
A VIEW is a big, detailed region where encoded data is shown. A GLYPH is a small iconic object that has internal structure built from several marks.

The honest answer is that there is no strict dividing line. The same little chart might be a view on its own or a glyph inside a bigger layout, depending on scale.

What matters is not the label but the consequence: at glyph size, only coarse patterns survive, so simplify the encoding accordingly.

IN ONE LINE

Scale changes what an encoding can carry. Design for the size it will be shown at.



Ten rules from tonight

Two weeks of material, compressed. Photograph this one.

- 1 Count your key columns first. That decides what you can draw.
- 2 Separate, order and align. Sorting a bar chart is almost always worth it.
- 3 Lines mean the points connect. Only use them when that is true.
- 4 Stack only when the parts genuinely sum to the total.
- 5 Check the aspect ratio before you conclude anything from a line chart.
- 6 Bending axes into a circle costs accuracy. Pay it only for cyclic data.
- 7 If one view cannot serve two tasks, use two views — or let the user switch.
- 8 Eyes beat memory: juxtapose for comparison, animate only smooth local change.
- 9 Keep hover under 0.1s and clicks under 1s, or it feels broken.
- 10 Design for the reader who never clicks. Most of them never do.

These are what Assignment 2 is graded against.

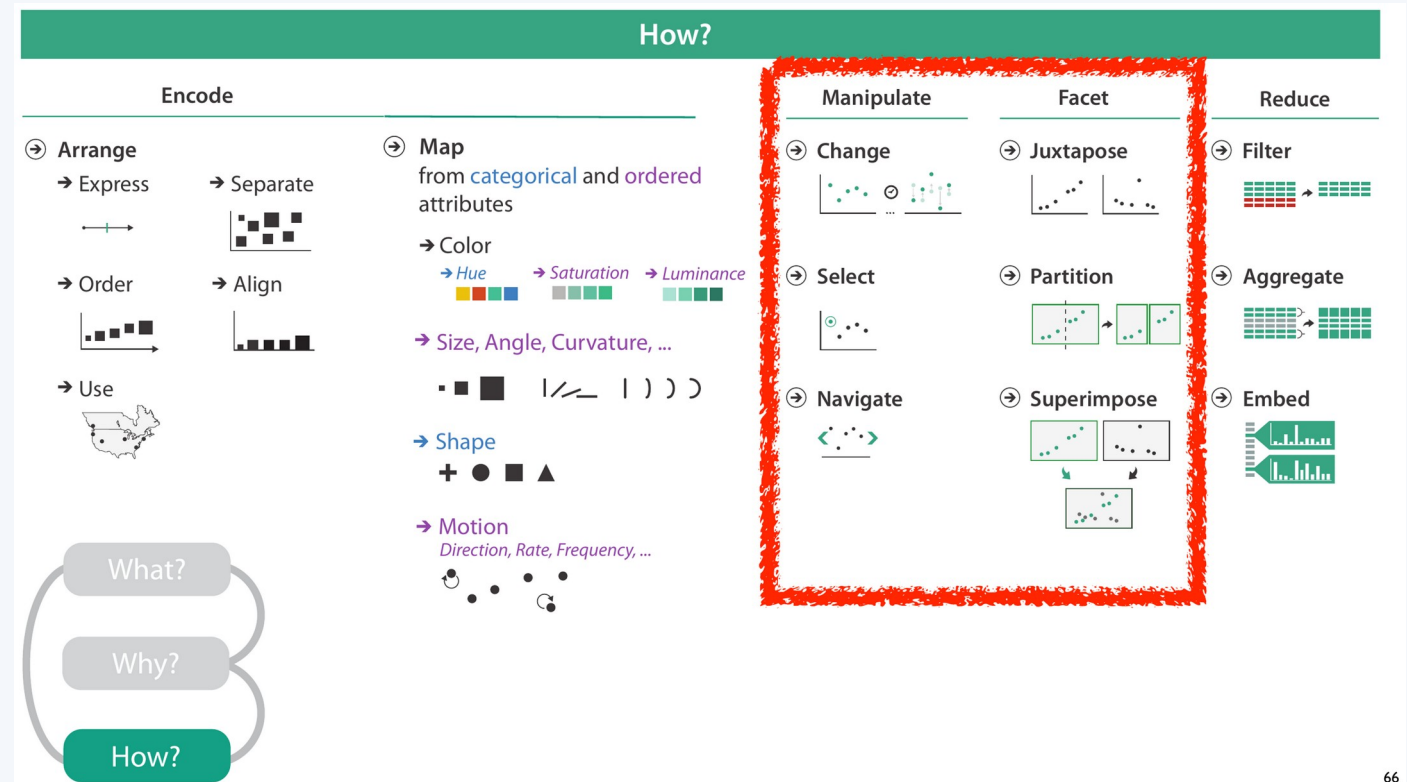
The complete HOW vocabulary

Everything under the HOW question, on one page.

Encode is Weeks 2 and 3 — marks, channels, and arrangement. Manipulate and Facet are tonight's second half. Reduce is Week 6.

Between this page and the Week 3 arrangement poster, you can describe the construction of any visualization you are likely to meet.

Both are posted full size on Blackboard.



Munzner, *Visualization Analysis and Design*, Ch. 11-12 — figures via the SPU Information Visualization course deck (M. Senturk), after Alex Lex & Miriah Meyer, dataviscourse.net.

Assignment 2 — and what to bring to Week 5

1 Read VAD Ch. 7, 11 and 12

Arrange Tables, Manipulate View, and Facet into Multiple Views.

Three chapters, but they are shorter than Ch. 5. These are heavily represented on the midterm.

2 Do Assignment 2

Posted on Blackboard with the full prompt and due date.

You will critique an interactive visualization using tonight's vocabulary — arrangement first, then interaction.

3 Find a linked-view example

Bring one real dashboard or interactive graphic that uses linked highlighting.

We will take it apart in Week 5 as a warm-up before Tableau.

4 Tableau must be installed

Week 5 begins in Tableau, hands on.

If your licence has not come through, tell me this week rather than next Monday.

5 Midterm is Week 7

It covers Weeks 1 through 4 — so everything up to tonight.

Start revising now rather than in Week 6, when the project also starts.

6 Think about project data

The project starts in Week 6. The hardest part is still finding a dataset you care about.

Bring candidates to office hours.

Next week: colour, and the start of Tableau.

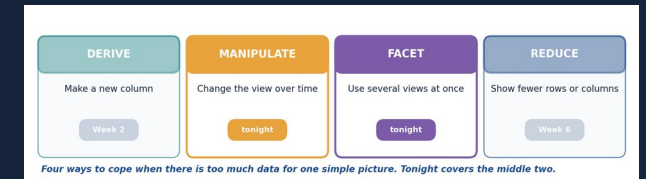
WEEKS 3 & 4 COMPLETE

Arrangement is the most powerful choice you have.

Position is the most accurate channel there is, and arranging a table is how you spend it. Everything tonight — including all of the interaction — is about getting more out of that one decision.

QUESTIONS

Reda MASTOURI · rmastouri@saintpeters.edu · 1 (888) 772-9933
Office hours and the discussion board are on Blackboard.



Three of the four complexity strategies now covered. Reduce arrives in Week 6.

Sources and image credits

PRIMARY TEXT

Munzner, T. Visualization Analysis and Design. A K Peters/CRC Press. ISBN 9781466508910. Companion site: cs.ubc.ca/~tmm/vadbook/ Chapter 7 ("Arrange Tables"), Chapter 11 ("Manipulate View") and Chapter 12 ("Facet into Multiple Views") are the reading behind this deck.

COURSE MATERIALS

Information Visualization course decks — Arrange Tables and Interactive Views — by Metin Senturk, MS in Data Science, Saint Peter's University (github.com/spu-dataviz-211), which credit Munzner VAD, Alex Lex & Miriah Meyer (dataviscourse.net), and Ben Jones (UW/Tableau).

RESEARCH CITED

Cleveland & McGill, Graphical Perception, JASA 79:387 (1984). · Zacks & Tversky, Bars and lines, Memory & Cognition 27(6), 1999. · Skau & Kosara, Arcs, Angles or Areas, Computer Graphics Forum, 2016. · Diehl, Beck & Burch, Uncovering strengths and weaknesses of radial visualizations, IEEE TVCG 2010. · Heer & Robertson, Animated Transitions in Statistical Data Graphics, IEEE TVCG 13(6), 2007. · Javed, McDonnell & Elmqvist, Graphical Perception of Multiple Time Series, IEEE TVCG 16:6, 2010. · Cockburn, Karlson & Bederson, A Review of Overview+Detail, ACM Computing Surveys 41:1, 2008. · Slingsby, Dykes & Wood (HIVE), IEEE TVCG 15:6, 2009. · Gratzl et al. (LineUp), IEEE TVCG 19:12, 2013. · Barsky et al. (Cerebral), IEEE TVCG 14:6, 2008. · Wills (EDV), NTTS 1995. · Jones, Harrold & Stasko (Tarantula), ICSE 2002.

EXAMPLES & FIGURES

Ben Jones / Tableau Public (bar ordering, dual axes, indexed lines, Gantt, slopegraphs, Poe). · Vox / Americans United for Life (Planned Parenthood chart). · flowingdata.com (box office streamgraph). · Mike Bostock, bl.ocks.org (sortable bars, collapsible tree, stacked-to-grouped, zoom to bounding box, zoomable icicle, brush & zoom). · carlmanaster.github.io/datastripes. · laurenwood.github.io (Growth of a Nation). · NYT (2013 budget proposal, oil prices); Bloomberg (What's Warming the World). · ralphstraumann.ch; buckets.peterbeshai.com. · Highcharts master-detail demo. · Stone, Get it right in black and white (stonesc.com). · Nightingale, Diagram of the Causes of Mortality, 1858.

ORIGINAL FIGURES

The roadmap, keys ladder, separate/order/align demonstration, bar-versus-line panels, aspect-ratio demonstration, axis-orientation comparison, pie-slice-count demonstration, stacked-versus-grouped comparison, layout-density demonstration, the manipulate and facet trios, the responsiveness budgets, the coordinated-views matrix and the four complexity strategies were generated for this deck.